SYSTEM DESIGN DOCUMENT

Design and Development of a Topic-Specific Web Crawler and Search Engine for Kenyan Job Listings

Submitted by Group:

Alvin Mwaura

Bachelor of Business and Information Technology (BBIT)

St. Paul's University

Course

BCS 3107 – Object-Oriented Systems Analysis and Design

Lecturer

Submission Date

Table of Contents

List of Figures

List of Tables

1. Introduction

1.1 Background

The rapid growth of internet technologies has resulted in an enormous increase in online information. Every day, thousands of new web pages are created, containing valuable information related to employment, education, healthcare, entertainment, and business. While this growth has increased access to information, it has also made it difficult for users to locate relevant content efficiently. Search engines address this challenge by using **web crawlers**, also known as spiders or bots, to automatically browse websites, collect information, and build searchable indexes.

A **web crawler** is a software application that systematically navigates web pages by following hyperlinks, downloading webpage content, extracting relevant information, and storing it in a searchable database. Modern search engines such as Google and Bing rely on sophisticated web crawlers to continuously update their search indexes. In addition to general-purpose search engines, organizations increasingly develop **topic-specific crawlers** to collect focused information from selected websites. Examples include job aggregators, price comparison systems, academic search platforms, and news monitoring services.

In Kenya, job opportunities are published across numerous company websites, recruitment agencies, government portals, and online job boards. Because these opportunities are distributed across different platforms, job seekers often spend significant time searching multiple websites to identify relevant vacancies. This fragmented approach can lead to missed opportunities, duplicated effort, and inefficient job searches.

This project proposes the design of a **Topic-Specific Web Crawler and Search Engine for Kenyan Job Listings**. The system automatically visits selected employment websites, extracts job-related information, indexes the collected data, and provides users with a centralized platform for searching available opportunities. The design follows **Object-Oriented Analysis and Design (OOAD)** principles, emphasizing modularity, abstraction, encapsulation, inheritance, and the use of interfaces and design patterns to produce a scalable and maintainable software architecture.

1.2 Problem Statement

Job seekers in Kenya are required to visit multiple websites to search for employment opportunities. Each website presents information differently, making it difficult to compare vacancies or perform comprehensive searches. Manual searching is time-consuming, repetitive, and prone to human error. Additionally, duplicate job advertisements frequently appear across different platforms, further reducing search efficiency.

Existing general-purpose search engines index a broad range of internet content but do not always prioritize locally relevant employment information. Consequently, users may struggle to locate current vacancies efficiently.

The absence of a centralized, topic-specific search platform creates several challenges:

- Time wasted searching multiple websites.
- Duplicate job advertisements.
- Difficulty identifying the most recent vacancies.
- Inconsistent search experiences.
- Limited ability to prioritize trusted sources.

To address these challenges, this project proposes a web crawler capable of collecting, filtering, indexing, and presenting job advertisements from selected Kenyan employment websites in a single searchable platform.

1.3 Project Objectives

General Objective

To design a topic-specific web crawler and search engine that automatically collects, indexes, and provides searchable access to Kenyan job listings using object-oriented design principles.

Specific Objectives

The project aims to:

1. Design a crawler that accepts multiple seed URLs.
2. Implement a URL frontier for crawl scheduling and prioritization.
3. Ensure compliance with website **robots.txt** and politeness policies.
4. Separate webpage links from downloadable file links.
5. Prevent duplicate webpage indexing.
6. Store indexed information in a relational database.
7. Provide efficient keyword-based search functionality.
8. Demonstrate the application of OOAD concepts through UML models and design patterns.

1.4 Scope

The scope of this project is limited to the analysis and design of a topic-specific web crawler targeting Kenyan job listing websites. The proposed system will include components for URL

management, webpage crawling, HTML parsing, duplicate detection, indexing, database storage, and keyword search. The project focuses on producing a complete system design rather than a fully implemented software application.

The design excludes advanced features such as distributed crawling, machine learning-based ranking algorithms, multilingual indexing, and cloud-native deployment, which are considered potential future enhancements.

2. Root Cause Analysis (RCA)

2.1 Overview

Root Cause Analysis (RCA) is a systematic problem-solving technique used to identify the underlying causes of a problem rather than addressing only its symptoms. In this project, RCA is applied to understand why job seekers experience difficulties when searching for employment opportunities online. Identifying these root causes provides a strong justification for the proposed web crawler system and guides the system requirements and design decisions.

2.2 Problem Statement for RCA

Problem: Job seekers spend excessive time searching multiple websites for employment opportunities and often encounter duplicate, outdated, or incomplete job information.

Symptoms

- Long search times.
- Duplicate job advertisements.
- Missed employment opportunities.
- Inconsistent website interfaces.
- Difficulty comparing vacancies from different sources.
- Inefficient manual searching.

2.3 Root Cause Analysis (Fishbone Categories)

The causes of the problem can be grouped into the following categories:

Technology

- Lack of centralized indexing.
- Inconsistent website structures.
- Different data formats across websites.
- Absence of intelligent crawling mechanisms.

Process

- Manual searching across multiple websites.
- No automated collection of vacancies.
- Delayed updates of job information.

People

- Users lack awareness of all available job websites.
- Human error when manually searching.
- Difficulty monitoring multiple sources regularly.

Data

- Duplicate advertisements.
- Outdated vacancies remain online.
- Missing job details on some websites.

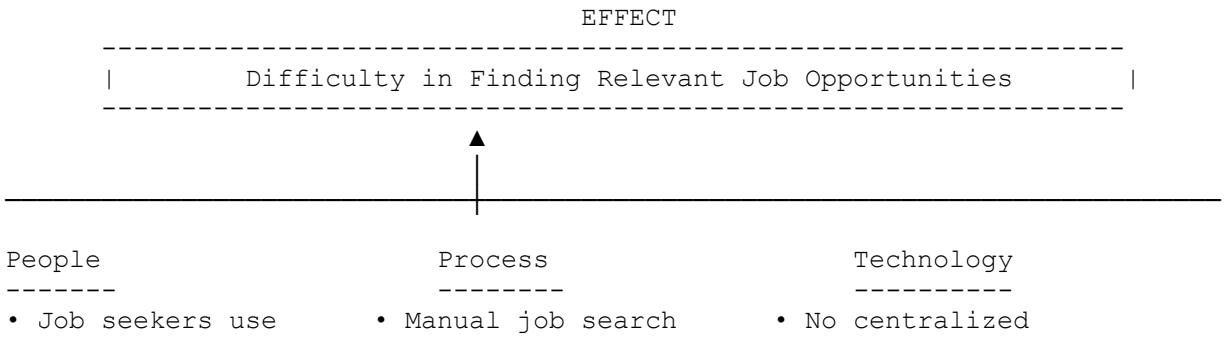
Environment

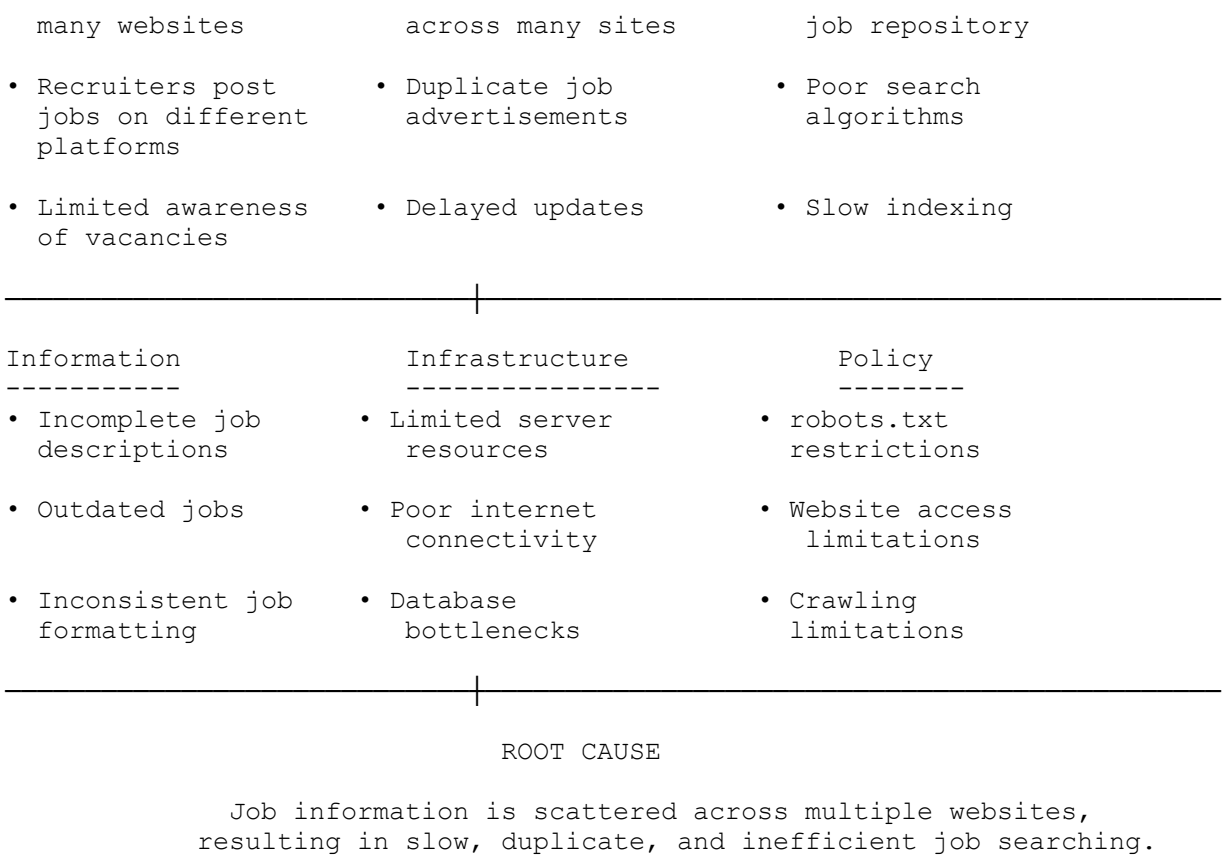
- Continuous growth in online job portals.
- Increasing competition for available positions.
- High volume of daily job postings.

2.4 Root Cause Discussion

The analysis shows that the primary cause of inefficient job searching is the absence of a centralized platform capable of automatically collecting and organizing employment information from multiple trusted websites. Instead of manually visiting each website, the proposed web crawler automates data collection, removes duplicate records, prioritizes trusted sources, and indexes job listings into a searchable database. This approach significantly reduces search time, improves information quality, and provides users with a more efficient and reliable job search experience.

Figure 2.1: Fishbone (Cause-and-Effect) Diagram





Caption

Figure 2.1: Fishbone (Cause-and-Effect) Diagram illustrating the root causes of inefficient online job searching, including issues related to people, processes, technology, information, infrastructure, and policy.

3. Market Analysis

3.1 Overview

Market analysis evaluates the need, demand, competition, and feasibility of the proposed **Topic-Specific Web Crawler and Search Engine for Kenyan Job Listings**. The purpose is to determine whether the system addresses a genuine problem, identify the target users, and assess how it compares with existing solutions.

The demand for online employment services in Kenya has grown significantly due to increasing internet access, smartphone usage, and digital recruitment practices. Organizations now advertise vacancies primarily through online platforms, making digital job search tools an essential resource for job seekers.

3.2 Current Market Situation

Currently, employment opportunities are scattered across numerous websites, including:

- Government recruitment portals
- Private company career pages
- Recruitment agencies
- University career portals
- Online job boards

Because information is distributed across many platforms, job seekers must repeatedly visit multiple websites to search for vacancies. This process is inefficient and increases the likelihood of missing suitable opportunities.

The proposed web crawler addresses this challenge by automatically collecting vacancies from trusted websites and presenting them through a centralized search interface.

3.3 Target Market

The primary users of the proposed system include:

a) Job Seekers

Individuals searching for employment opportunities across various industries.

Needs

- Fast job searching
- Accurate search results
- Up-to-date vacancies
- Simple interface

b) Employers

Organizations wishing to increase the visibility of their job advertisements.

Needs

- Wider audience reach
- Increased application rates
- Faster recruitment

c) Recruitment Agencies

Organizations responsible for sourcing and advertising vacancies.

Needs

- Centralized vacancy management
- Efficient candidate attraction
- Automated advertisement indexing

d) Researchers

Researchers analyzing employment trends and labor market data.

Needs

- Structured employment datasets
- Historical job records
- Statistical reporting

3.4 Competitor Analysis

Several platforms currently provide employment information. However, each has limitations that the proposed system seeks to address.

Competitor	Strengths	Weaknesses
LinkedIn Jobs	Large global network	Not focused on Kenyan jobs
BrighterMonday Kenya	Popular recruitment platform	Limited to jobs posted on its platform
MyJobMag Kenya	Good search interface	Does not aggregate jobs from multiple external websites
Corporate Career Pages	Official vacancies	Users must visit each website individually
Government Recruitment Portals	Authentic public sector jobs	Limited search functionality and fragmented information

The proposed system differentiates itself by automatically crawling selected trusted websites, removing duplicate listings, and providing a unified search experience.

3.5 SWOT Analysis

A SWOT analysis evaluates the internal and external factors affecting the proposed system.

Strengths	Weaknesses
Centralized job search	Initial setup complexity
Automated data collection	Requires periodic maintenance
Duplicate detection	Dependent on website availability
Scalable architecture	Limited to selected websites initially
Opportunities	**Threats**
Growing online recruitment	Website structure changes
Increased internet usage	Legal restrictions on web crawling
Expansion to regional job markets	Competition from established job platforms
Integration with AI-based recommendations	Changes in robots.txt policies

3.6 Feasibility Analysis

Technical Feasibility

The proposed system can be implemented using widely available technologies such as:

- Python
- Flask
- BeautifulSoup
- Scrapy
- MySQL
- Git and GitHub

The development tools are open source and suitable for implementing the system architecture.

Economic Feasibility

The project requires minimal financial investment because it relies primarily on open-source software and standard computing resources.

Estimated costs include:

- Development computer

- Internet connection
- Optional cloud hosting for deployment

No expensive software licenses are required.

Operational Feasibility

The proposed system is expected to improve job searching by providing users with:

- Faster access to vacancies
- Centralized job information
- Improved search efficiency
- Reduced duplication of job listings

The user interface is designed to be intuitive, minimizing training requirements.

Schedule Feasibility

The project can be completed within a standard university semester using an incremental development approach.

Proposed phases include:

1. Requirements Analysis
2. System Design
3. Database Design
4. Interface Design
5. Testing
6. Documentation

3.7 Benefits of the Proposed System

The proposed web crawler offers several benefits over existing approaches:

- Reduces the time required to search for jobs.
- Aggregates vacancies from multiple trusted websites.
- Eliminates duplicate job advertisements.
- Improves the accuracy and relevance of search results.
- Provides a centralized repository of employment opportunities.

- Supports future expansion into other domains such as scholarships, tenders, or real estate listings.

4. Stakeholder Analysis Using the MoSCoW Method

4.1 Overview

Stakeholder analysis identifies the individuals and organizations that influence or are affected by the proposed system. The **MoSCoW prioritization technique** is used to classify system requirements based on their importance.

MoSCoW categorizes requirements into four groups:

- **Must Have**
- **Should Have**
- **Could Have**
- **Won't Have (for this release)**

This prioritization ensures that critical features are implemented first while less essential features are scheduled for future versions.

4.2 Stakeholders

The primary stakeholders include:

- Job Seekers
- System Administrator
- Employers
- Recruitment Agencies
- University Researchers
- System Developers

Each stakeholder has different expectations and responsibilities within the system.

4.3 MoSCoW Prioritization

Must Have (Essential Requirements)

These requirements are mandatory for the successful operation of the system.

Requirement	Reason
Seed URL management	Starting point for web crawling
Web crawling	Core functionality
HTML parsing	Extract webpage content
Duplicate detection	Prevent multiple indexing of the same page
Database storage	Store indexed information
Keyword search	Enable users to search jobs
robots.txt compliance	Respect website policies
URL prioritization	Improve crawling efficiency
Administrator authentication	Secure system management

Should Have (Important Requirements)

These features improve usability and system performance but are not essential for the first release.

Requirement	Benefit
Crawl scheduling	Automated periodic crawling
Search filters	Improve search precision
Crawl statistics	Performance monitoring
Error logging	Simplifies troubleshooting
Backup and recovery	Protects indexed data

Could Have (Desirable Requirements)

These features add value but can be implemented in future releases.

Requirement	Benefit
Email job notifications	Notify users of new vacancies
AI-based job recommendations	Personalized search experience
Mobile application	Improved accessibility
Data analytics dashboard	Employment trend visualization
Cloud deployment	Increased scalability

Won't Have (Current Release)

These features are intentionally excluded from Version 1 due to project scope and time constraints.

Requirement	Reason
Distributed web crawling	Requires additional infrastructure
Machine learning ranking algorithms	Beyond current project scope
Natural language processing	Future enhancement
Multi-language support	Requires additional resources
Real-time collaboration	Not required for initial release

4.4 Stakeholder Responsibilities

Stakeholder	Responsibilities
Job Seekers	Search for jobs and provide feedback
Employers	Publish job opportunities on source websites
Recruitment Agencies	Advertise vacancies and maintain listings
System Administrator	Configure seed URLs, manage users, monitor system performance
Developers	Develop, test, and maintain the system
Researchers	Analyze employment data and generate reports

4.5 Stakeholder Benefits

The proposed system provides benefits to each stakeholder group.

Stakeholder	Benefits
Job Seekers	Faster and centralized access to job opportunities
Employers	Greater visibility of job advertisements
Recruitment Agencies	Wider reach for vacancy announcements
Administrators	Efficient monitoring and management of crawler operations
Researchers	Access to structured employment data for analysis

4.6 Conclusion

The market analysis demonstrates a clear need for a centralized job search platform capable of aggregating employment opportunities from multiple trusted websites. The stakeholder analysis

confirms that the proposed system addresses the needs of job seekers, employers, recruiters, administrators, and researchers. By prioritizing requirements using the MoSCoW method, the project focuses first on delivering the essential features required for a functional and effective topic-specific web crawler while allowing room for future enhancements.

5. Requirements Clarification

5.1 Overview

Requirements clarification is a critical phase in the Object-Oriented Systems Analysis and Design (OOAD) process. It involves identifying and documenting the functional and non-functional requirements that the proposed Topic-Specific Web Crawler and Search Engine must satisfy. Clearly defined requirements ensure that the system addresses stakeholder needs while providing a solid foundation for design, implementation, testing, and future maintenance.

The requirements for this project were gathered through analysis of existing job search platforms, stakeholder needs, and the project specification provided in the course assignment.

5.2 User Roles

The system consists of two primary user roles:

5.2.1 System Administrator

The System Administrator is responsible for configuring and managing the crawler.

Responsibilities

- Manage seed URLs.
- Configure crawler settings.
- Monitor crawling activities.
- View crawl reports.
- Manage indexed data.
- Manage user accounts.
- Schedule crawling operations.

5.2.2 Job Seeker

The Job Seeker is the end user of the system.

Responsibilities

- Search for jobs.
- Filter search results.
- View job details.
- Access recently indexed vacancies.

5.3 Functional Requirements

Functional requirements describe the services and operations that the system must perform.

FR1: User Authentication

Description

The administrator shall log into the system using secure credentials before accessing administrative functions.

FR2: Seed URL Management

Description

The administrator shall add, edit, delete, and manage multiple seed URLs from which crawling begins.

FR3: Web Crawling

Description

The system shall automatically crawl websites starting from the configured seed URLs.

FR4: Robots.txt Compliance

Description

The crawler shall check and obey the robots.txt file of every website before accessing webpages.

FR5: HTML Parsing

Description

The system shall extract hyperlinks, job titles, company names, job descriptions, deadlines, and locations from downloaded webpages.

FR6: Link Classification

Description

The crawler shall distinguish between:

- Web pages
- PDF files
- Images
- Documents
- Other downloadable files

Only HTML pages containing relevant job information shall be indexed.

FR7: URL Prioritization

Description

The system shall prioritize URLs according to predefined rules such as:

- Website importance
- Page freshness
- Crawl depth
- Keyword relevance

FR8: Duplicate Detection

Description

The system shall prevent indexing the same webpage more than once using URL normalization and content hashing techniques.

FR9: Indexing

Description

The crawler shall store extracted job information in a searchable database.

FR10: Keyword Search

Description

Users shall search indexed jobs using keywords.

Example:

```
Software Engineer Nairobi
```

The search engine shall return relevant vacancies ranked by relevance.

FR11: Reporting

The administrator shall generate reports showing:

- Number of crawled pages
- Indexed pages
- Failed crawls
- Duplicate pages
- Crawl duration

FR12: Crawl Scheduling

The administrator shall schedule automatic crawling.

Example:

- Daily
- Weekly
- Monthly

5.4 Functional Requirements Summary

Requirement ID	Description	Priority
FR1	User Authentication	Must
FR2	Seed URL Management	Must
FR3	Web Crawling	Must
FR4	Robots.txt Compliance	Must
FR5	HTML Parsing	Must
FR6	Link Classification	Must
FR7	URL Prioritization	Must
FR8	Duplicate Detection	Must
FR9	Indexing	Must
FR10	Keyword Search	Must
FR11	Reporting	Should
FR12	Crawl Scheduling	Should

5.5 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

Performance

The system shall:

- Crawl at least **1,000 webpages per hour**.
- Return search results within **2 seconds**.
- Support at least **500 concurrent users**.
- Maintain database response times below **500 milliseconds**.

Reliability

The system shall:

- Recover automatically after minor failures.
- Continue crawling even if individual webpages cannot be accessed.
- Maintain data consistency during indexing.

Availability

The crawler shall achieve at least **99% uptime** during scheduled operations.

Security

The system shall:

- Encrypt user passwords.
- Prevent SQL injection attacks.
- Prevent Cross-Site Scripting (XSS).
- Implement role-based access control.
- Validate all user input.

Scalability

The architecture shall support:

- Additional websites.
- Larger databases.
- Increased crawling frequency.
- Additional system users.

Usability

The user interface shall:

- Be intuitive and easy to navigate.
- Support responsive design.
- Display clear navigation menus.
- Provide meaningful error messages.

Maintainability

The system shall:

- Use modular object-oriented design.
- Follow coding standards.
- Be easy to update and extend.
- Support future enhancements.

Portability

The application shall run on:

- Windows
- Linux
- macOS

without requiring major modifications.

5.6 Use Case Summary

Actor	Use Case
Administrator	Login
Administrator	Add Seed URL
Administrator	Edit Seed URL
Administrator	Delete Seed URL
Administrator	Configure Crawl Settings
Administrator	Schedule Crawls
Administrator	View Reports
Job Seeker	Search Jobs
Job Seeker	View Job Details

Actor	Use Case
Job Seeker	Filter Search Results

5.7 Requirement Traceability Matrix (RTM)

Requirement	System Component	Test Case
FR1	Authentication Module	ST-01
FR2	URL Frontier	ST-02
FR3	Web Crawler	ST-03
FR4	Robots.txt Handler	ST-04
FR5	HTML Parser	ST-05
FR6	Link Classifier	ST-06
FR7	Priority Queue	ST-07
FR8	Duplicate Detector	ST-08
FR9	Indexer	ST-09
FR10	Search Engine	ST-10
FR11	Reporting Module	ST-11
FR12	Scheduler	ST-12

5.8 Requirement Validation

The identified requirements were validated against the project objectives and stakeholder expectations.

Validation Criterion	Status
Requirements are complete	✓
Requirements are consistent	✓
Requirements are testable	✓
Requirements are achievable	✓
Requirements align with stakeholder needs	✓
Requirements satisfy assignment specifications	✓

5.9 Chapter Summary

This chapter defined the functional and non-functional requirements of the proposed Topic-Specific Web Crawler and Search Engine. The system requirements directly address the project objectives by ensuring the crawler can manage seed URLs, respect website policies, prioritize crawling, classify links, eliminate duplicate indexing, and provide efficient keyword-based searching. These requirements form the basis for the architectural and detailed system design presented in the following chapters.

Next Chapter (Chapter 6): Back-of-the-Envelope Estimation

The next section will include:

- Storage estimation
- Bandwidth estimation
- Number of websites/pages to crawl
- Database size calculations
- Crawl rate calculations
- Server resource estimation
- Capacity planning

This will closely follow the system design methodology required in your assignment.

6. Back-of-the-Envelope Estimation

6.1 Overview

Back-of-the-envelope estimation is a high-level system design technique used to estimate the resources required for implementing and operating a software system. These estimates help determine the expected storage requirements, processing capacity, network bandwidth, and overall scalability of the proposed Topic-Specific Web Crawler and Search Engine.

Although the figures presented are estimates, they provide a realistic basis for designing the system architecture and planning future expansion.

6.2 Assumptions

To simplify the calculations, the following assumptions are made:

Parameter	Value
Number of seed websites	20

Parameter	Value
Average pages per website	5,000
Total webpages to crawl	100,000
Average HTML page size	500 KB
Average crawl frequency	Once per day
Average extracted text per page	30 KB
Average hyperlinks per page	50
Concurrent users	500
Search requests per day	15,000
Average search response time	Less than 2 seconds

These assumptions represent a medium-scale web crawler suitable for a university project and can be adjusted for larger deployments.

6.3 Estimation of Pages to Crawl

The crawler starts from the configured seed URLs and follows hyperlinks within the allowed domains.

Formula

Total Pages=Seed Websites×Average Pages per Website\text{Total Pages} = \text{Seed Websites} \times \text{Average Pages per Website}Total Pages=Seed Websites×Average Pages per Website

Calculation

20 × 5,000 = **100,000 webpages**

Therefore, the crawler is expected to index approximately **100,000 webpages**.

6.4 Storage Estimation

The system stores only the extracted job information rather than the complete HTML content for long-term indexing.

Average Extracted Data per Page

Data	Estimated Size
Job Title	100 Bytes
Company Name	80 Bytes
Location	100 Bytes
Description	25 KB
URL	300 Bytes
Metadata	4 KB

Average record size ≈ **30 KB**

Storage Calculation

100,000 pages × 30 KB

= **3,000,000 KB**

≈ **3 GB**

Additional Database Indexes

Allow approximately **20% overhead** for indexes and metadata.

3 GB × 20%

= **0.6 GB**

Total Estimated Database Storage

≈ 3.6 GB

6.5 Bandwidth Estimation

Average webpage size:

500 KB

Total pages:

100,000

Formula

Bandwidth = Page Size × Number of Pages

Calculation

500 KB × 100,000

= 50,000,000 KB

≈ **50 GB**

Since crawling occurs once daily,

Daily bandwidth requirement ≈ **50 GB/day**

Monthly bandwidth:

50 GB × 30

= **1.5 TB/month**

6.6 Crawl Rate Estimation

The system should complete crawling within approximately **24 hours**.

Pages

100,000

Time

24 Hours

Formula

Pages per Hour

100,000 ÷ 24

≈ **4,167 pages/hour**

Pages per Minute

4,167 ÷ 60

≈ **69 pages/minute**

Pages per Second

69 ÷ 60

≈ **1.15 pages/second**

This crawl rate is achievable using asynchronous crawling and multiple worker threads.

6.7 Search Query Estimation

Expected daily searches

15,000

Average searches per hour

15,000 ÷ 24

≈ **625 searches/hour**

Average searches per minute

625 ÷ 60

≈ **10 searches/minute**

Peak traffic may reach approximately **40 searches per minute**, which the system should handle efficiently.

6.8 Database Estimation

Estimated number of indexed jobs

100,000

Estimated users

10,000

Estimated searches stored

500,000 annually

Estimated crawl logs

300,000 annually

Table	Estimated Records
Users	10,000
Seed URLs	500
Crawled Pages	100,000
Indexed Jobs	100,000
Search History	500,000
Crawl Logs	300,000

Total estimated records:

Approximately **1 million records**

A relational database such as MySQL or PostgreSQL can efficiently manage this volume.

6.9 Memory Estimation

The crawler maintains a queue of URLs during execution.

Average queue size:

5,000 URLs

Average URL size:

200 Bytes

Memory required:

5,000 × 200 Bytes

= **1 MB**

Parser cache:

≈ **200 MB**

Crawler workers:

≈ **600 MB**

Application memory:

≈ **1 GB**

Recommended RAM:

4–8 GB

6.10 Server Estimation

A single mid-range server is sufficient for the proposed system.

Resource	Recommended Specification
CPU	Quad-Core Processor
RAM	8 GB
Storage	100 GB SSD
Database	MySQL
Operating System	Ubuntu Server 22.04 LTS
Network	100 Mbps

This configuration supports crawling, indexing, searching, and administrative functions for the estimated workload.

6.11 Scalability Estimation

The system is designed to support future growth.

Potential scalability improvements include:

- Adding additional crawler workers.
- Deploying distributed crawling nodes.
- Using load balancers.
- Migrating to cloud infrastructure.

- Implementing database replication.
- Integrating caching mechanisms such as Redis.

These enhancements would allow the system to crawl millions of webpages while maintaining acceptable performance.

6.12 Summary of Estimates

Metric	Estimated Value
Seed Websites	20
Total Pages	100,000
Average Page Size	500 KB
Daily Bandwidth	50 GB
Monthly Bandwidth	1.5 TB
Database Size	3.6 GB
Search Requests per Day	15,000
Concurrent Users	500
Crawl Speed	4,167 pages/hour
Recommended RAM	8 GB
Recommended Storage	100 GB SSD

6.13 Chapter Summary

The back-of-the-envelope estimation demonstrates that the proposed Topic-Specific Web Crawler can operate efficiently with modest hardware resources while meeting the expected workload. The calculations indicate that approximately **100,000 webpages** can be crawled daily, requiring **3.6 GB** of database storage and **50 GB** of daily bandwidth. An **8 GB RAM**, **quad-core processor**, and **100 GB SSD** server provide sufficient capacity for the proposed deployment. These estimates form the basis for the logical and physical architecture presented in the following chapter.

7. System Overview

7.1 Overview

The **Topic-Specific Web Crawler and Search Engine** is designed to automate the collection, processing, indexing, and retrieval of job advertisements from selected Kenyan websites. The

system follows the principles of **Object-Oriented Analysis and Design (OOAD)** by organizing functionality into independent, reusable, and maintainable components. Each component has a well-defined responsibility and interacts with other components through clearly defined interfaces, making the system modular, scalable, and easy to maintain.

The architecture consists of three primary layers:

1. **Presentation Layer** – Provides user interfaces for administrators and job seekers.
2. **Application Layer** – Handles business logic such as crawling, parsing, indexing, and searching.
3. **Data Layer** – Stores system data, indexed pages, and user information.

This layered approach promotes **separation of concerns**, allowing each layer to evolve independently without affecting the others.

7.2 High-Level System Architecture

The system begins with the administrator configuring one or more **seed URLs**. These URLs are placed into a **URL Frontier**, which manages the order in which pages are crawled. The **Web Crawler** downloads webpages while checking each site's **robots.txt** file to ensure compliance with crawling policies.

Downloaded HTML pages are passed to the **HTML Parser**, which extracts job information and hyperlinks. The extracted hyperlinks are classified into webpage links and downloadable file links. Only valid webpage links are sent back to the URL Frontier for future crawling.

The extracted job data is then checked by the **Duplicate Detector**, which prevents indexing of repeated content. Unique records are forwarded to the **Indexer**, where they are processed and stored in the database. Users can search indexed job listings through the **Search Engine**, which retrieves relevant information from the database.

Figure 7.1: Overall System Architecture
(Insert System Architecture Diagram here.)

7.3 Major System Components

The proposed system consists of the following major components.

7.3.1 User Interface

The User Interface provides interaction between users and the system.

Functions

- Administrator login
- Seed URL management
- Crawl monitoring
- Report generation
- Job search interface
- View indexed job details

7.3.2 Authentication Module

The Authentication Module verifies administrator credentials before granting access to system management features.

Responsibilities

- User login
- Password verification
- Session management
- Access control

7.3.3 URL Frontier

The URL Frontier maintains a queue of URLs waiting to be crawled.

Responsibilities

- Store seed URLs
- Prioritize URLs
- Remove processed URLs
- Prevent duplicate URLs in the queue

The URL Frontier uses a **Priority Queue** data structure to improve crawling efficiency.

7.3.4 Robots.txt Manager

Before crawling a webpage, the Robots.txt Manager checks whether crawling is permitted.

Responsibilities

- Download robots.txt
- Verify crawling permissions
- Apply crawl delays
- Block restricted URLs

This ensures compliance with website policies and ethical web crawling practices.

7.3.5 Web Crawler

The Web Crawler retrieves webpages from the internet.

Responsibilities

- Download HTML pages
- Handle network errors
- Retry failed requests
- Pass HTML content to the parser

The crawler processes only webpages permitted by the Robots.txt Manager.

7.3.6 HTML Parser

The HTML Parser extracts useful information from downloaded webpages.

Extracted Information

- Job title
- Company name
- Location
- Job description
- Application deadline
- Hyperlinks
- Metadata

The parser separates webpage links from file links such as PDFs and images.

7.3.7 Duplicate Detector

Duplicate detection prevents the same webpage from being indexed multiple times.

Techniques Used

- URL normalization
- Content hashing
- Unique URL comparison

This improves storage efficiency and prevents duplicate search results.

7.3.8 Indexer

The Indexer processes extracted information before storing it in the database.

Responsibilities

- Create searchable indexes
- Remove unnecessary HTML tags
- Store structured job information
- Update existing records

7.3.9 Search Engine

The Search Engine allows users to retrieve indexed job information using keywords.

Features

- Keyword search
- Search ranking
- Result filtering
- Sorting by relevance
- Pagination

Example search:

```
Software Engineer Nairobi
```

7.3.10 Database

The database stores all persistent system information.

Stored Data

- Users
- Seed URLs
- Crawled pages
- Indexed jobs
- Search history
- Crawl logs

7.4 Component Iteraction

The components interact in the following sequence:

1. Administrator adds seed URLs.
2. URLs are stored in the URL Frontier.
3. The Web Crawler retrieves the next URL.
4. Robots.txt Manager verifies crawling permission.
5. Web Crawler downloads the webpage.
6. HTML Parser extracts job information and hyperlinks.
7. Duplicate Detector checks for existing records.
8. Unique records are indexed.
9. Data is stored in the database.
10. Users search indexed jobs through the Search Engine.

7.5 Object-Oriented Design Principles Applied

The system applies several core OOAD principles.

Encapsulation

Each class manages its own data and operations. For example, the `WebCrawler` class handles webpage downloading without exposing internal implementation details.

Abstraction

Complex operations such as crawling, parsing, and indexing are represented through high-level interfaces, simplifying interaction between components.

Inheritance

Specialized classes can inherit common behavior from base classes. For example, different parser implementations can extend a generic `Parser` class.

Polymorphism

Multiple parser implementations can process different webpage structures while exposing a common interface.

Modularity

Each component performs a single responsibility, making the system easier to develop, test, and maintain.

7.6 Design Patterns Used

Several software design patterns improve flexibility and maintainability.

Design Pattern	Purpose
Singleton	Ensures a single database connection manager.
Factory	Creates parser objects for different webpage types.
Strategy	Supports multiple URL prioritization algorithms.
Observer	Notifies monitoring components when crawling events occur.

7.7 System Workflow

The overall workflow of the system is illustrated below:

1. Administrator logs into the system.
2. Seed URLs are configured.
3. URL Frontier schedules crawling.
4. Web Crawler downloads webpages.
5. Robots.txt Manager validates permissions.
6. HTML Parser extracts job information.
7. Duplicate Detector filters repeated pages.
8. Indexer processes and stores unique records.
9. Search Engine retrieves relevant job listings.
10. Users view search results and job details.

7.8 Advantages of the Proposed Architecture

The architecture offers several benefits:

- **Scalability:** New crawler nodes or data sources can be added with minimal changes.
- **Maintainability:** Modular components simplify updates and bug fixes.
- **Performance:** URL prioritization and indexing improve crawl and search efficiency.
- **Reliability:** Error handling and duplicate detection enhance system stability.
- **Security:** Authentication and input validation protect system resources.
- **Extensibility:** Additional features such as AI-based recommendations or cloud deployment can be integrated in future versions.

7.9 Chapter Summary

This chapter presented the overall architecture of the proposed Topic-Specific Web Crawler and Search Engine. The system is organized into presentation, application, and data layers, with dedicated components for crawling, parsing, indexing, searching, and administration. By applying object-oriented principles and established design patterns, the architecture achieves modularity, scalability, and maintainability. The next chapter expands on this foundation by providing the detailed system architecture, UML models, and component interactions that describe how the design is implemented.

8. System Architecture

8.1 Overview of the System Architecture

The proposed **Topic-Specific Web Crawler and Search Engine** adopts a **three-tier architecture** comprising the Presentation Layer, Application Layer, and Data Layer. This

architecture separates user interaction, business logic, and data storage, making the system easier to develop, maintain, and scale.

The architecture supports the complete web crawling lifecycle, from receiving seed URLs to crawling webpages, extracting job information, indexing content, and providing search functionality to users.

Figure 8.1: Overall System Architecture

(Insert System Architecture Diagram here.)

=

8.2 Architectural Layers

8.2.1 Presentation Layer

The Presentation Layer provides the interface through which users interact with the system.

Components

- Administrator Dashboard
- Login Page
- Job Search Page
- Search Results Page
- Reports Dashboard

Responsibilities

- Display information
- Accept user input
- Authenticate users
- Display search results
- Display crawl statistics

8.2.2 Application Layer

The Application Layer contains all business logic.

Components

- Authentication Manager

- URL Frontier
- Scheduler
- Robots.txt Manager
- Web Crawler
- HTML Parser
- Link Classifier
- Duplicate Detector
- Indexer
- Search Engine
- Reporting Module

Responsibilities

- Crawl webpages
- Process HTML
- Remove duplicates
- Build search indexes
- Handle search requests
- Generate reports

8.2.3 Data Layer

The Data Layer manages permanent storage.

Database Tables

- Users
- Seed URLs
- Crawl Queue
- Crawled Pages
- Indexed Jobs
- Search History
- Crawl Logs

Responsibilities

- Store crawled data
- Store user accounts
- Maintain indexes
- Support search queries
- Store reports

8.3 System Workflow

The following sequence describes how the system operates.

Step 1

Administrator logs into the system.

↓

Step 2

Administrator adds Seed URLs.

↓

Step 3

Seed URLs enter the URL Frontier.

↓

Step 4

Scheduler selects the next URL.

↓

Step 5

Robots.txt Manager verifies crawling permission.

↓

Step 6

Web Crawler downloads webpage.

↓

Step 7

HTML Parser extracts:

- Job Title
- Company
- Description
- Location
- Links

↓

Step 8

Link Classifier separates

- HTML Pages
- PDF Files
- Images
- Documents

↓

Step 9

Duplicate Detector checks whether the page already exists.

↓

Step 10

Indexer processes unique pages.

↓

Step 11

Database stores indexed information.

↓

Step 12

Search Engine retrieves matching jobs.

↓

Step 13

Job seeker views results.

8.4 System Components

8.4.1 Authentication Module

Purpose

Controls administrator access.

Inputs

- Username
- Password

Outputs

- Login success
- Login failure

8.4.2 URL Frontier

Purpose

Maintains URLs waiting for crawling.

Inputs

- Seed URLs
- Newly discovered URLs

Outputs

- Next prioritized URL

8.4.3 Scheduler

Purpose

Controls crawling order.

Responsibilities

- Schedule crawling
- Prevent excessive requests
- Prioritize URLs

8.4.4 Robots.txt Manager

Purpose

Ensures ethical crawling.

Responsibilities

- Read robots.txt
- Verify permissions
- Apply crawl delay

8.4.5 Web Crawler

Purpose

Downloads webpages.

Inputs

- URL

Outputs

- HTML Document

8.4.6 HTML Parser

Purpose

Extract useful information.

Outputs

- Job title
- Company
- Description
- Deadline
- Location
- Hyperlinks

8.4.7 Link Classifier

Purpose

Separate webpage links from downloadable files.

Example

Web Pages

```
https://company.com/jobs
```

Files

```
jobs.pdf
application.docx
poster.jpg
```

Only HTML pages continue through the crawling process.

8.4.8 Duplicate Detector

Purpose

Prevent duplicate indexing.

Techniques

- URL comparison
- Content hashing
- Canonical URL matching

8.4.9 Indexer

Responsibilities

- Build searchable indexes
- Remove unnecessary HTML
- Save structured data

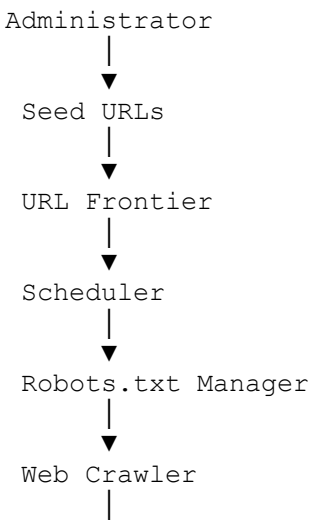
8.4.10 Search Engine

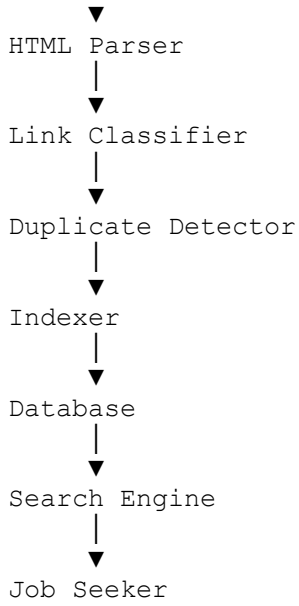
Responsibilities

- Search keywords
- Rank results
- Filter jobs
- Sort by relevance

8.5 Data Flow

The flow of data through the system is summarized below:





This logical flow should correspond to your **DFD Level 0** and **System Architecture Diagram**.

8.6 Object-Oriented Architecture

The proposed system follows OOAD principles.

Encapsulation

Each class protects its internal data.

Examples:

- WebCrawler
- HTMLParser
- SearchEngine

Abstraction

Complex crawling operations are hidden behind simple interfaces.

Example:

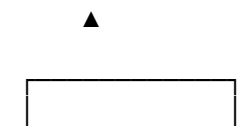
```
Crawler.start()
```

instead of exposing download logic.

Inheritance

Example hierarchy

```
Crawler
```



```
NewsCrawler JobCrawler
```

Polymorphism

Different parser implementations process different websites while using the same interface.

Example

```
Parser.parse()
```

Each website-specific parser implements this method differently.

Interfaces

Example interfaces

```
CrawlerInterface
```

```
ParserInterface
```

```
SearchInterface
```

Interfaces reduce coupling between modules.

8.7 Architectural Design Patterns

Pattern	Usage
Singleton	Database Connection

Pattern	Usage
Factory	Create HTML Parsers
Strategy	URL Prioritization
Observer	Crawl Monitoring
MVC	Overall application architecture

These patterns improve extensibility, maintainability, and code reuse.

8.8 Component Interaction

The interaction among components is summarized below.

Component	Sends Data To
Administrator	URL Frontier
URL Frontier	Scheduler
Scheduler	Web Crawler
Web Crawler	HTML Parser
HTML Parser	Link Classifier
Link Classifier	Duplicate Detector
Duplicate Detector	Indexer
Indexer	Database
Search Engine	Database
Job Seeker	Search Engine

8.9 Architecture Advantages

The architecture provides several advantages:

- Modular design with clearly defined responsibilities.
- Easy integration of additional crawlers or search features.
- High scalability through layered architecture.
- Improved maintainability due to separation of concerns.
- Better security through authentication and access control.
- Efficient crawling using prioritization and duplicate detection.
- Future-ready design supporting cloud deployment and distributed crawling.

8.10 Chapter Summary

This chapter described the overall architecture of the Topic-Specific Web Crawler and Search Engine, including its layered design, major components, data flow, and object-oriented principles. The architecture demonstrates how the system achieves modularity, scalability, and maintainability while satisfying the project requirements. The application of design patterns such as **Singleton**, **Factory**, **Strategy**, **Observer**, and **MVC** further enhances flexibility and code reuse. The next chapter presents the **Detailed Design**, including UML diagrams, class relationships, sequence interactions, activity flows, and component-level implementation details.

9. Detailed Design

9.1 Overview

The Detailed Design chapter translates the system architecture into implementable software components. It specifies the internal structure of the system using **Unified Modeling Language (UML)** diagrams and object-oriented design principles. Each module is described in terms of its responsibilities, relationships, operations, and interactions.

The proposed Topic-Specific Web Crawler and Search Engine follows a modular architecture where each component performs a specific function while communicating with other components through well-defined interfaces. This approach improves maintainability, scalability, and code reuse.

9.2 UML Use Case Design

The Use Case Diagram identifies the interactions between system users (actors) and the system functions.

Actors

- System Administrator
- Job Seeker

Administrator Use Cases

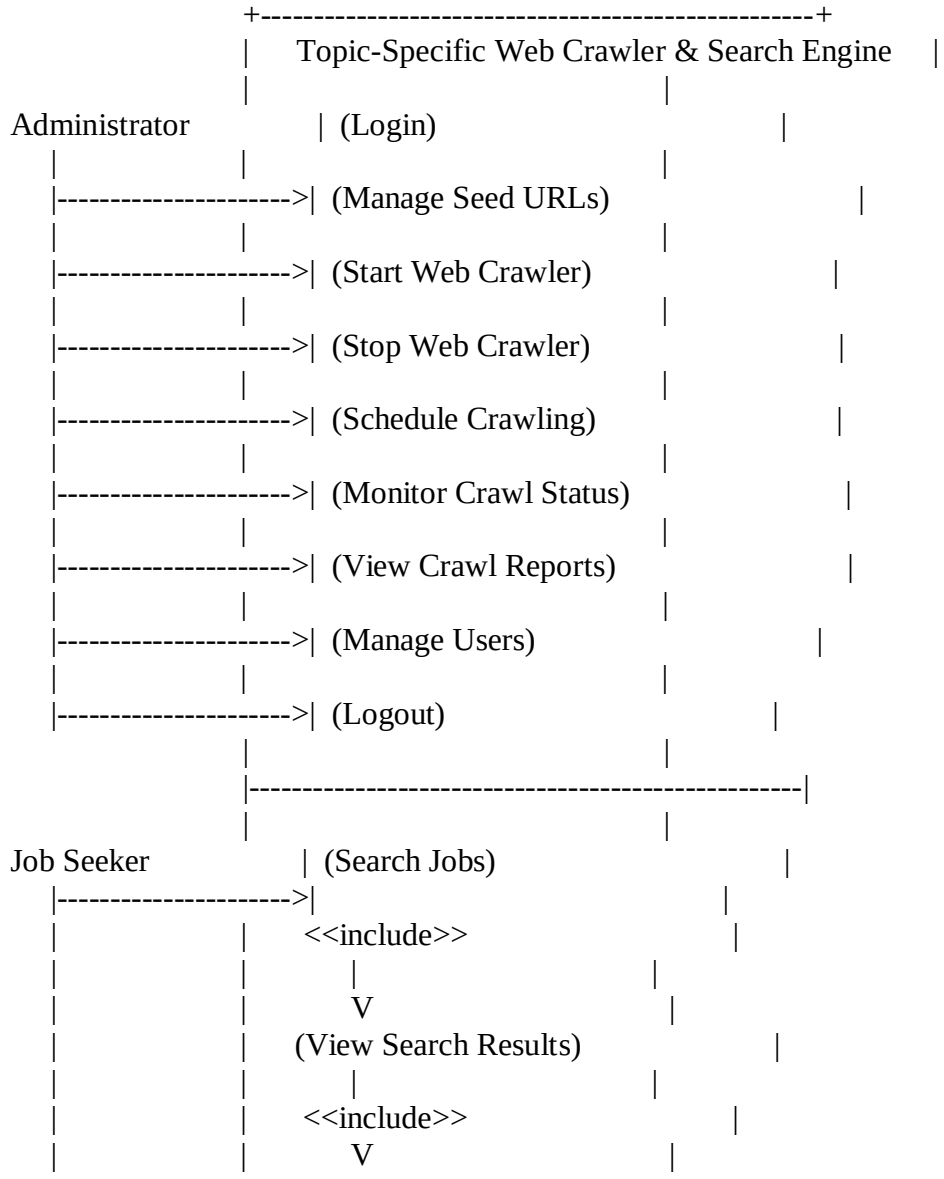
- Login
- Add Seed URL
- Edit Seed URL
- Delete Seed URL
- Configure Crawl Settings

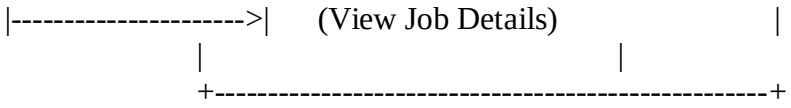
- Start Crawl
- Stop Crawl
- Schedule Crawl
- View Crawl Reports
- Manage Users

Job Seeker Use Cases

- Search Jobs
- Filter Results
- View Job Details
- View Recent Jobs

Figure 9.1: Use Case Diagram





9.3 UML Class Design

The Class Diagram models the static structure of the system by defining classes, attributes, methods, and relationships.

Main Classes

User

Attributes

- userID
- username
- password
- role

Methods

- login()
- logout()

Administrator

Attributes

- adminID

Methods

- addSeedURL()
- scheduleCrawler()
- generateReport()

Administrator inherits from the **User** class.

URLFrontier

Attributes

- queue
- priority

Methods

- enqueue()
- dequeue()
- prioritize()

WebCrawler

Attributes

- crawlerID
- currentURL

Methods

- crawl()
- downloadPage()

HTMLParser

Methods

- extractLinks()
- extractJobDetails()
- classifyLinks()

DuplicateDetector

Methods

- isDuplicate()
- generateHash()

Indexer

Methods

- createIndex()
- updateIndex()

SearchEngine

Methods

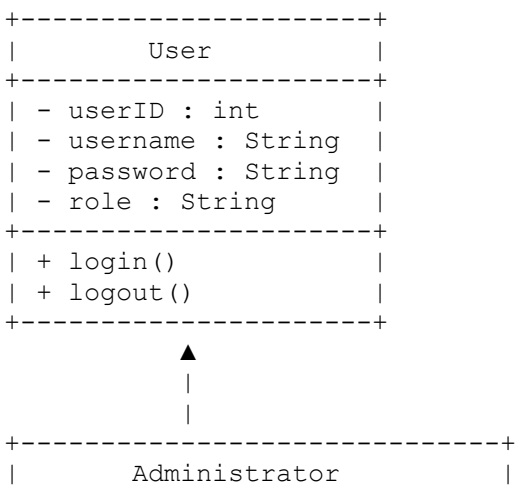
- search()
- rankResults()

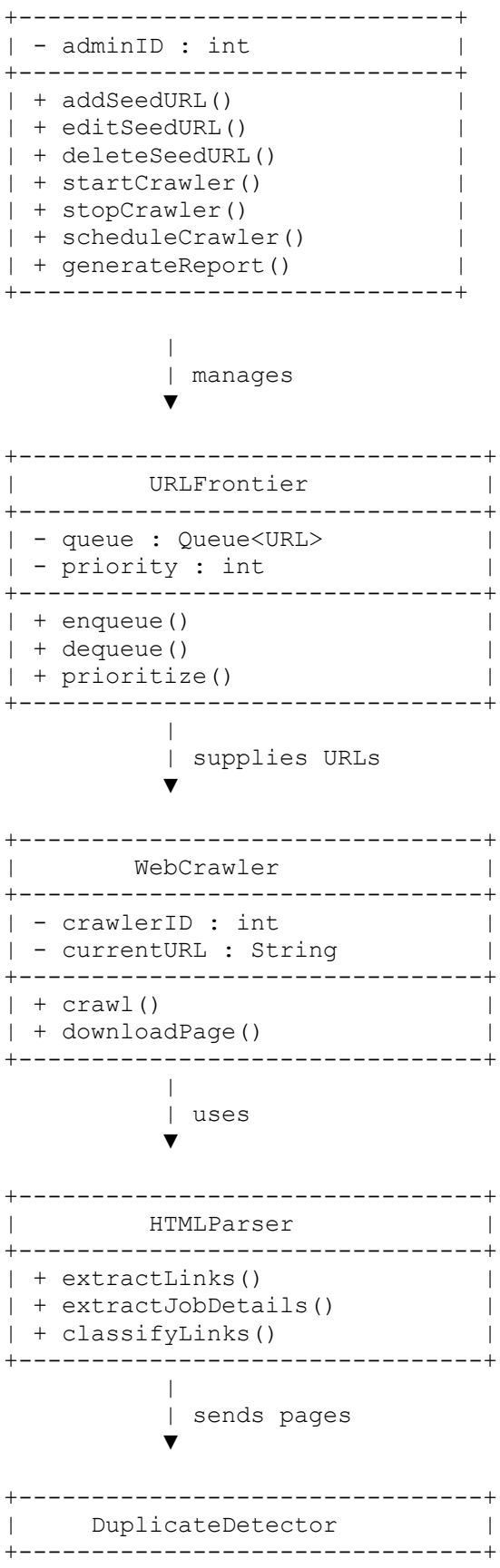
DatabaseManager

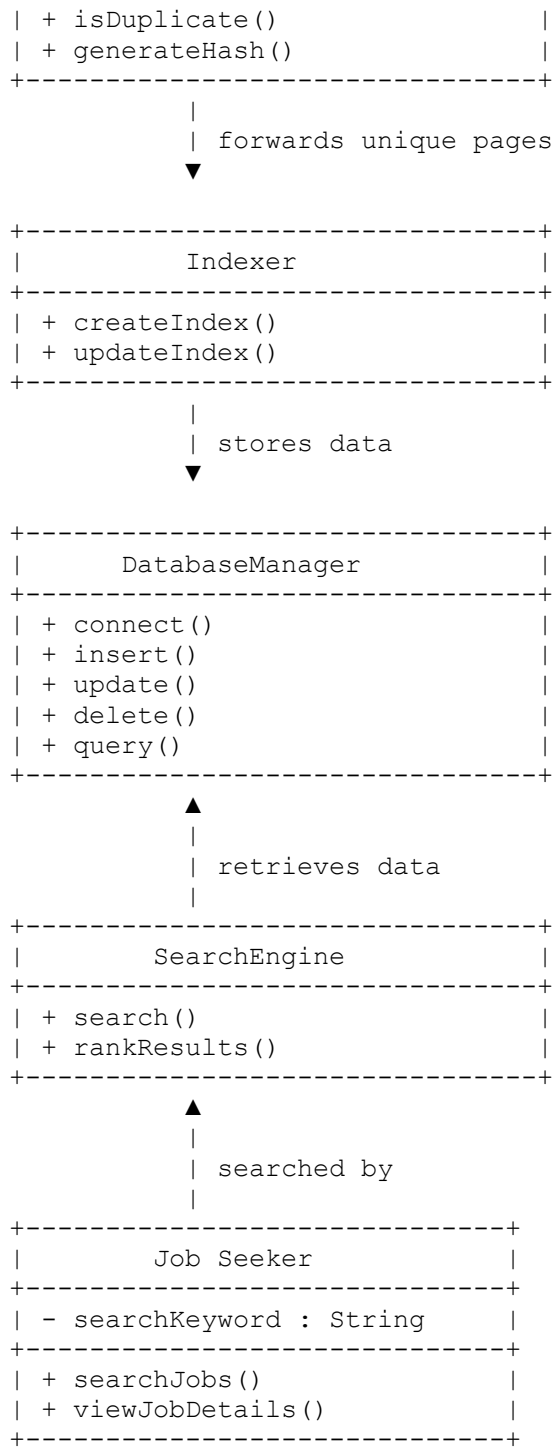
Methods

- connect()
- insert()
- update()
- delete()
- query()

Figure 9.2: UML Class Diagram




- adminID : int
+ addSeedURL()
+ editSeedURL()
+ deleteSeedURL()
+ startCrawler()
+ stopCrawler()
+ scheduleCrawler()
+ generateReport()
manages
URLFrontier
- queue : Queue<URL>
- priority : int
+ enqueue()
+ dequeue()
+ prioritize()
supplies URLs
WebCrawler
- crawlerID : int
- currentURL : String
+ crawl()
+ downloadPage()
uses
HTMLParser
+ extractLinks()
+ extractJobDetails()
+ classifyLinks()
sends pages
DuplicateDetector



Relationships

Relationship	Type
`Administrator` inherits from `User`	Inheritance
`Administrator` manages `URLFrontier`	Association

Relationship	Type
`URLFrontier` provides URLs to `WebCrawler`	Association
`WebCrawler` uses `HTMLParser`	Association
`HTMLParser` sends data to `DuplicateDetector`	Association
`DuplicateDetector` forwards unique pages to `Indexer`	Association
`Indexer` stores data in `DatabaseManager`	Association
`SearchEngine` retrieves data from `DatabaseManager`	Association
`Job Seeker` uses `SearchEngine`	Association

- **Figure Caption**
- **Figure 9.2:** UML Class Diagram illustrating the main classes, attributes, methods, and relationships within the Topic-Specific Web Crawler and Search Engine

9.4 Class Relationships

The classes interact using standard object-oriented relationships.

Relationship	Description
Inheritance	Administrator inherits from User
Association	WebCrawler uses HTMLParser
Association	HTMLParser communicates with DuplicateDetector
Association	DuplicateDetector communicates with Indexer
Association	SearchEngine retrieves records from DatabaseManager
Aggregation	URLFrontier stores multiple URLs
Dependency	Scheduler depends on URLFrontier

9.5 UML Sequence Design

The Sequence Diagram illustrates the interaction between objects during a typical crawling operation.

Scenario

Administrator starts a crawling session.

Sequence

1. Administrator logs in.
2. Administrator starts crawler.
3. Scheduler requests next URL.
4. URLFrontier returns URL.
5. WebCrawler downloads webpage.
6. HTMLParser extracts data.
7. DuplicateDetector checks uniqueness.
8. Indexer stores page.
9. Database saves record.
10. Administrator receives crawl status.

Figure 9.3: Sequence Diagram
(Insert Sequence Diagram here.)

9.6 UML Activity Design

The Activity Diagram illustrates the workflow followed during crawling.

Activity Flow

Start

↓

Administrator enters Seed URL

↓

Validate URL

↓

Check robots.txt

↓

Permission Granted?

- Yes → Continue crawling
- No → Skip page

↓

Download HTML

↓

Extract hyperlinks

↓

Extract job details

↓

Check duplicate

↓

Duplicate?

- Yes → Ignore
- No → Index page

↓

Store in Database

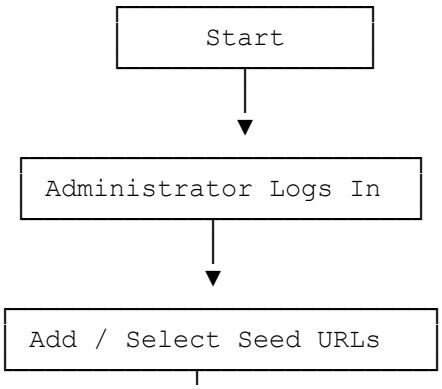
↓

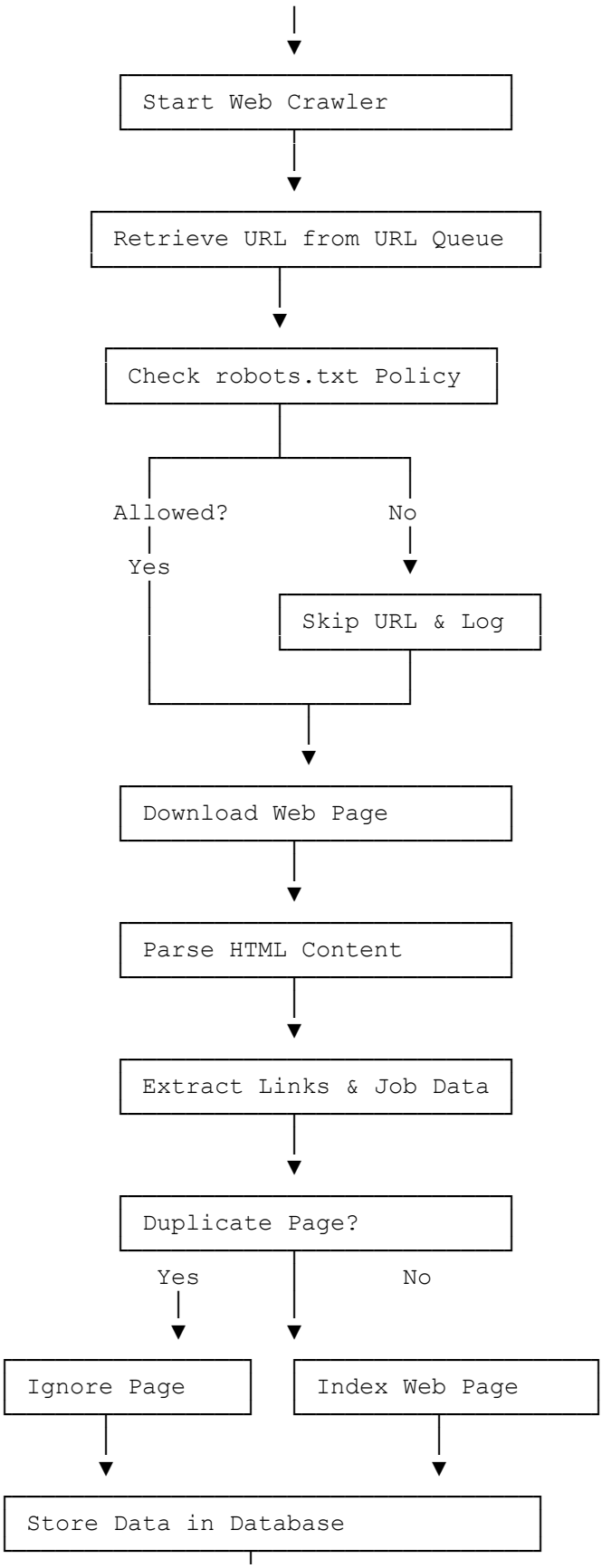
Repeat until Queue Empty

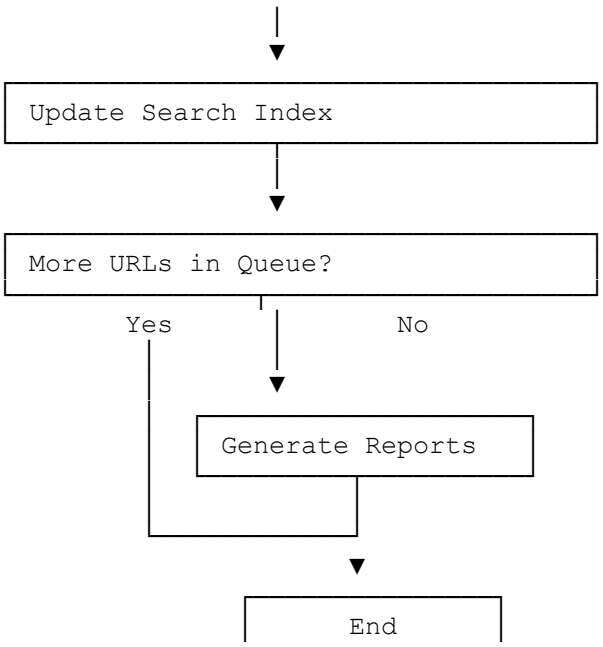
↓

End

Figure 9.4: Activity Diagram




Start Web Crawler
Retrieve URL from URL Queue
Check robots.txt Policy
Allowed?
Yes
No
Skip URL & Log
Download Web Page
Parse HTML Content
Extract Links & Job Data
Duplicate Page?
Yes
No
Ignore Page
Index Web Page
Store Data in Database



9.7 UML Component Design

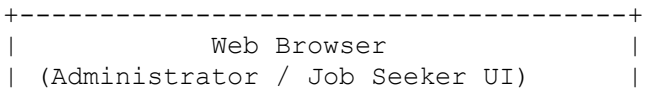
The Component Diagram illustrates the logical organization of the software modules.

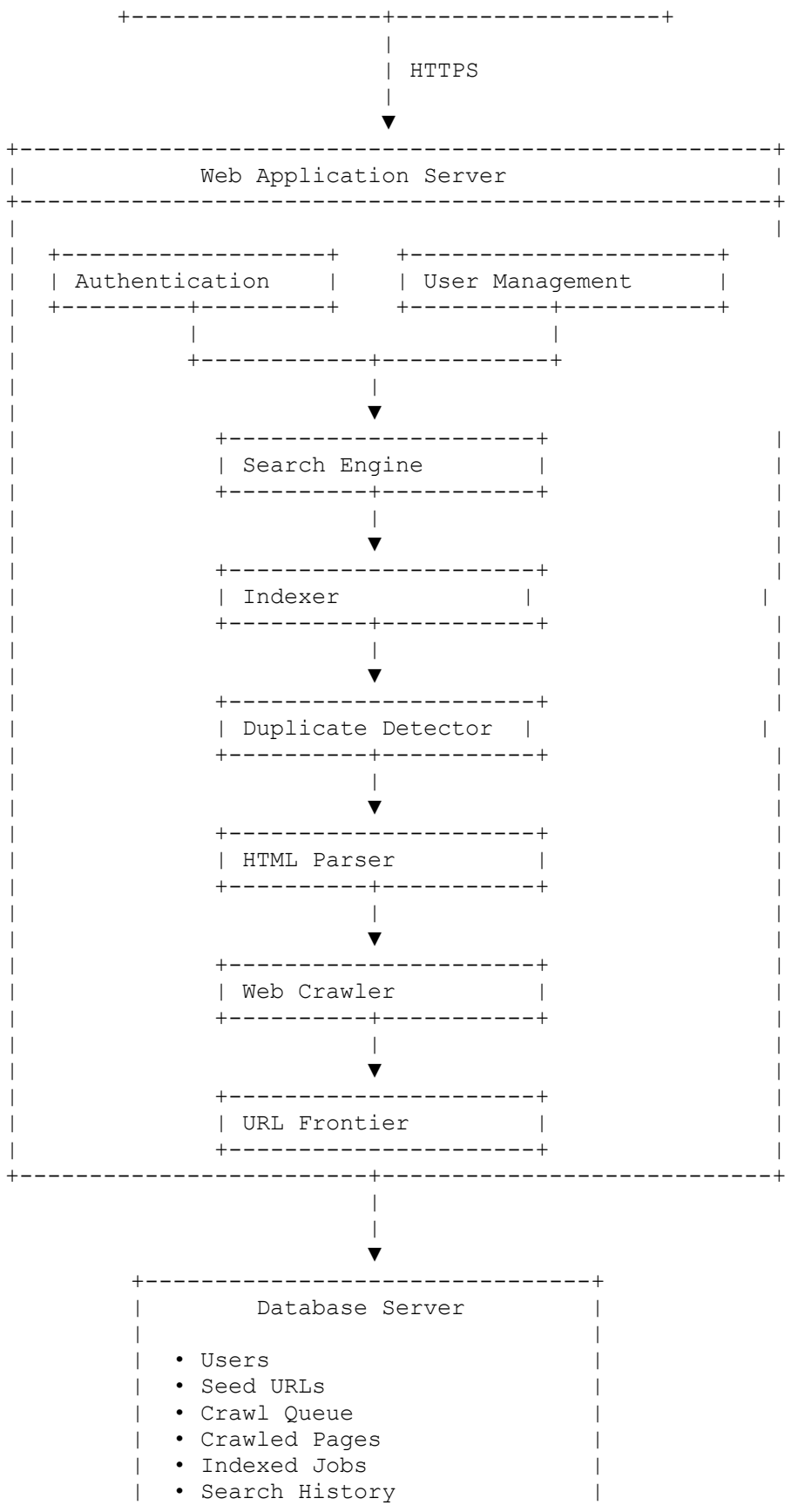
Components

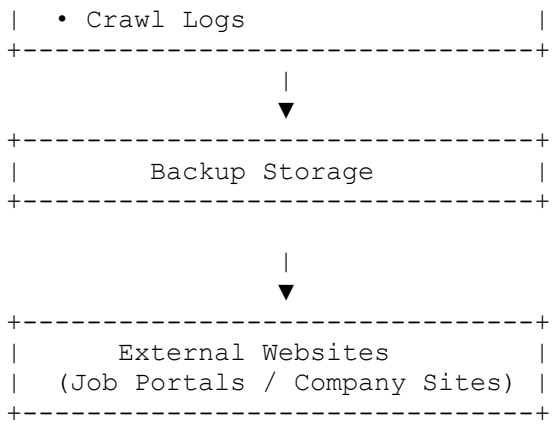
- Web Interface
- Authentication Module
- URL Frontier
- Scheduler
- Robots.txt Manager
- Web Crawler
- HTML Parser
- Duplicate Detector
- Indexer
- Search Engine
- Reporting Module
- Database

Each component exposes interfaces that allow communication with other components while maintaining low coupling.

Figure 9.5: Component Diagram




HTTPS
Web Application Server
Authentication
User Management
Search Engine
Indexer
Duplicate Detector
HTML Parser
Web Crawler
URL Frontier
Database Server
• Users
• Seed URLs
• Crawl Queue
• Crawled Pages
• Indexed Jobs
• Search History



Component Descriptions

Component	Description
Web Browser	Interface used by administrators and job seekers.
Authentication	Handles login, logout, and session management.
User Management	Manages administrator accounts and roles.
Search Engine	Processes user search queries and retrieves ranked results.
Indexer	Creates and updates the searchable index of crawled pages.
Duplicate Detector	Identifies and removes duplicate webpages before indexing.
HTML Parser	Extracts hyperlinks and job information from downloaded pages.
Web Crawler	Downloads webpages from external websites while respecting crawling policies.
URL Frontier	Maintains the prioritized queue of URLs to crawl.
Database Server	Stores all persistent system data, including users, pages, jobs, and logs.
Backup Storage	Maintains scheduled backups for disaster recovery.
External Websites	Source websites from which job listings are crawled.

Component Relationships

- The **Web Browser** communicates with the **Web Application Server** over **HTTPS**.

- **Authentication** verifies users before granting access.
- The **Web Crawler** retrieves pages from **External Websites**.
- The **HTML Parser** extracts relevant content from downloaded pages.
- The **Duplicate Detector** filters out duplicate pages.
- The **Indexer** stores processed data in the **Database Server**.
- The **Search Engine** queries the database to provide search results.
- The **Database Server** periodically backs up data to **Backup Storage**.

Figure Caption

Figure 9.5: UML Component Diagram showing the major software components of the Topic-Specific Web Crawler and Search Engine and their interactions.

9.8 UML Deployment Design

The Deployment Diagram illustrates how software components are distributed across hardware devices.

Client Device

- Web Browser
- Administrator
- Job Seeker

↓

Application Server

- Web Application
- Authentication Service
- Crawler Engine
- Search Engine

↓

Database Server

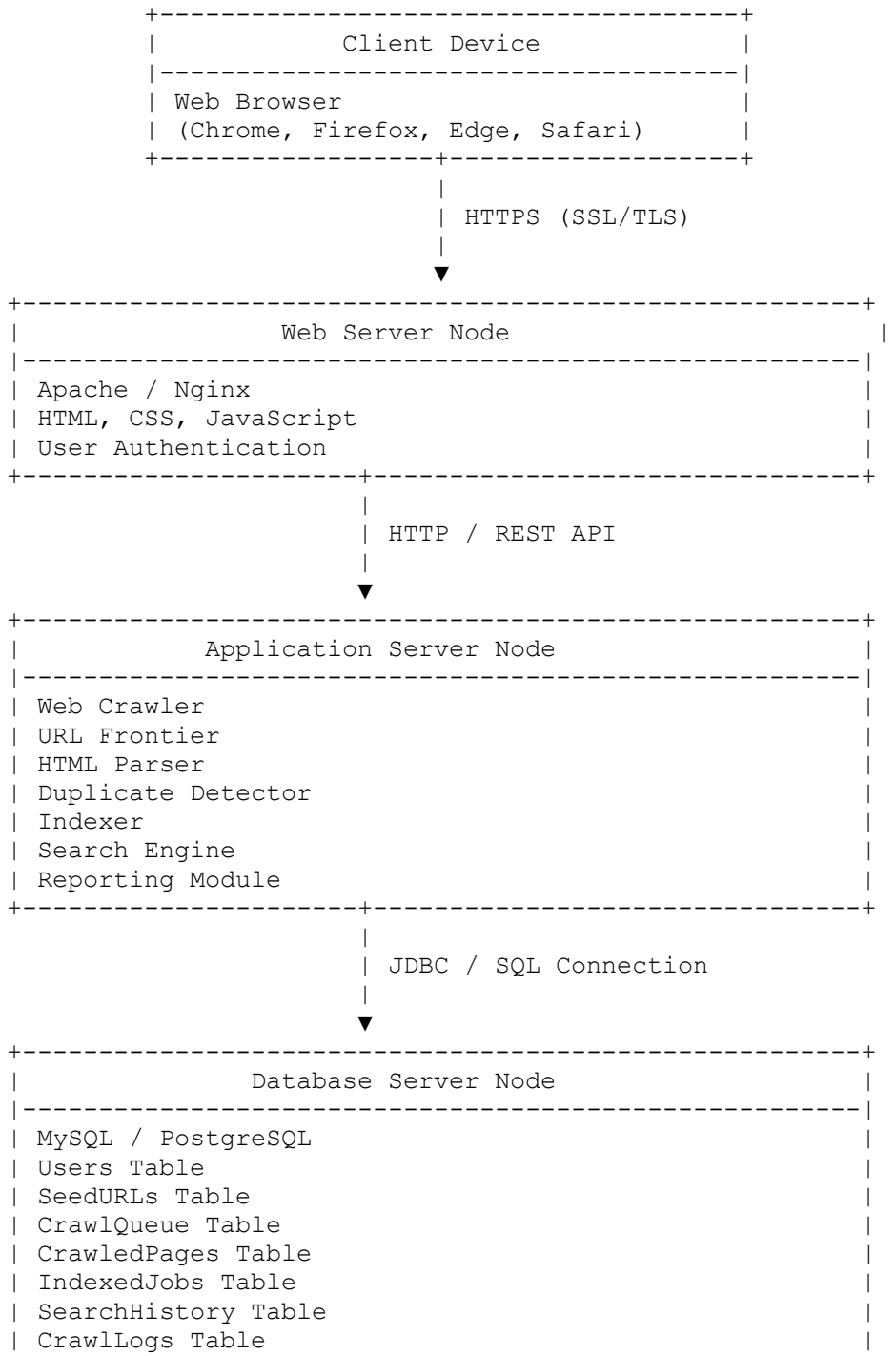
- MySQL Database

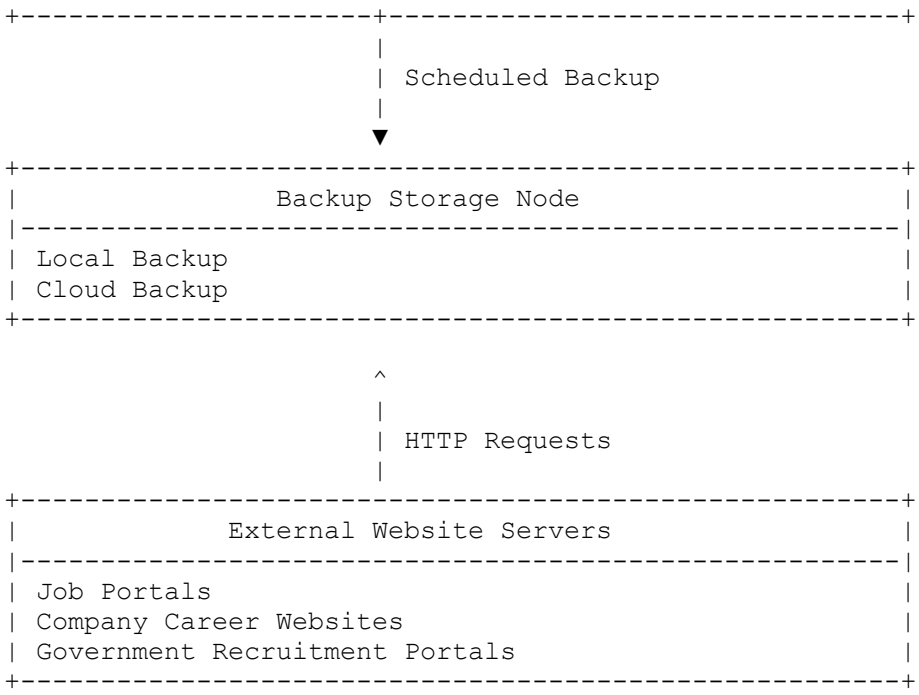
↓

Internet

- Target Websites
- robots.txt Files

Figure 9.6: Deployment Diagram





9.9 Object-Oriented Design Principles

The design applies key object-oriented principles.

Encapsulation

Each class hides its internal data and exposes only necessary methods.

Example:

- `WebCrawler`
- `SearchEngine`

Abstraction

Complex processes are simplified through interfaces.

Example:

```
Crawler.start();
```

Inheritance

Inheritance reduces code duplication.

Example:

```
User
   |
Administrator
```

Polymorphism

Multiple parser classes implement the same interface.

Example:

```
Parser.parse();
```

Different websites may use different parsing logic while maintaining the same method signature.

Interfaces

Interfaces reduce dependency among modules.

Examples:

- `CrawlerInterface`
- `ParserInterface`
- `SearchInterface`

9.10 Design Patterns

The following software design patterns are incorporated into the system.

Design Pattern	Purpose	Example
Singleton	Single database connection	DatabaseManager
Factory	Creates parser objects	ParserFactory
Strategy	URL prioritization algorithms	PriorityStrategy
Observer	Crawl notifications	CrawlMonitor

Design Pattern	Purpose	Example
MVC	Separates UI, business logic, and data	Entire application

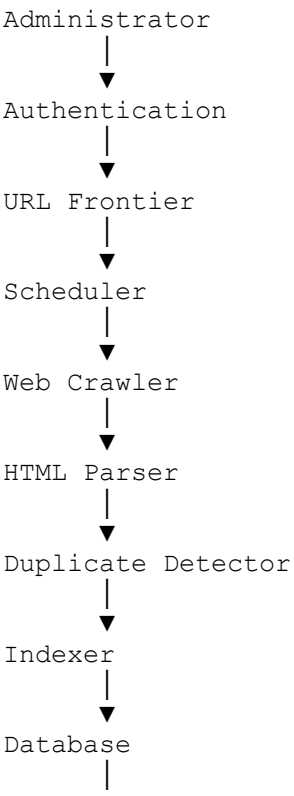
9.11 Exception Handling Design

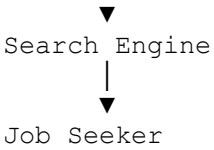
The system handles unexpected situations gracefully.

Exception	Handling Strategy
Network Timeout	Retry up to three times
robots.txt Restricted	Skip page and log event
Invalid URL	Reject and notify administrator
Database Connection Failure	Reconnect automatically
Duplicate Page	Ignore duplicate and continue
HTML Parsing Error	Log error and continue crawling

9.12 Module Dependency Diagram

The dependency hierarchy is as follows:





This sequence illustrates how each module depends on the previous one while maintaining a clear separation of responsibilities.

9.13 Chapter Summary

This chapter presented the detailed design of the Topic-Specific Web Crawler and Search Engine using UML models and object-oriented principles. The design includes use case, class, sequence, activity, component, and deployment diagrams, along with descriptions of class relationships, design patterns, and exception handling strategies. These design artefacts provide a comprehensive blueprint for implementing the system while ensuring modularity, maintainability, and scalability. The next chapter focuses on the **Data Design**, including the database schema, entity relationships, and data flow required to support the system's functionality.

10. Data Design

10.1 Overview

The Data Design defines how information is structured, stored, accessed, and managed within the proposed **Topic-Specific Web Crawler and Search Engine**. A well-designed database ensures efficient storage of crawled webpages, indexed job listings, user accounts, search history, and crawler logs while maintaining data integrity and minimizing redundancy.

The proposed system uses a **Relational Database Management System (RDBMS)** such as **MySQL** or **PostgreSQL** because these databases provide reliable transaction management, indexing, and support for structured queries.

10.2 Database Design Objectives

The database is designed to achieve the following objectives:

- Store crawled webpages efficiently.
- Eliminate duplicate records.
- Maintain data integrity.

- Support fast keyword searches.
- Store administrator accounts securely.
- Record crawler activities and logs.
- Support future scalability.

10.3 Database Entities

The proposed database consists of the following major entities.

Entity	Purpose
Users	Stores administrator information
SeedURLs	Stores initial crawling URLs
CrawlQueue	Stores URLs waiting to be crawled
CrawledPages	Stores downloaded webpages
IndexedJobs	Stores extracted job advertisements
SearchHistory	Stores user search requests
CrawlLogs	Stores crawler execution logs

=

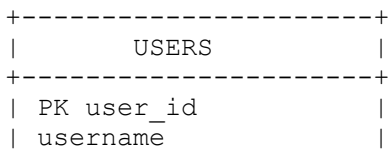
10.4 Entity Relationship Diagram (ERD)

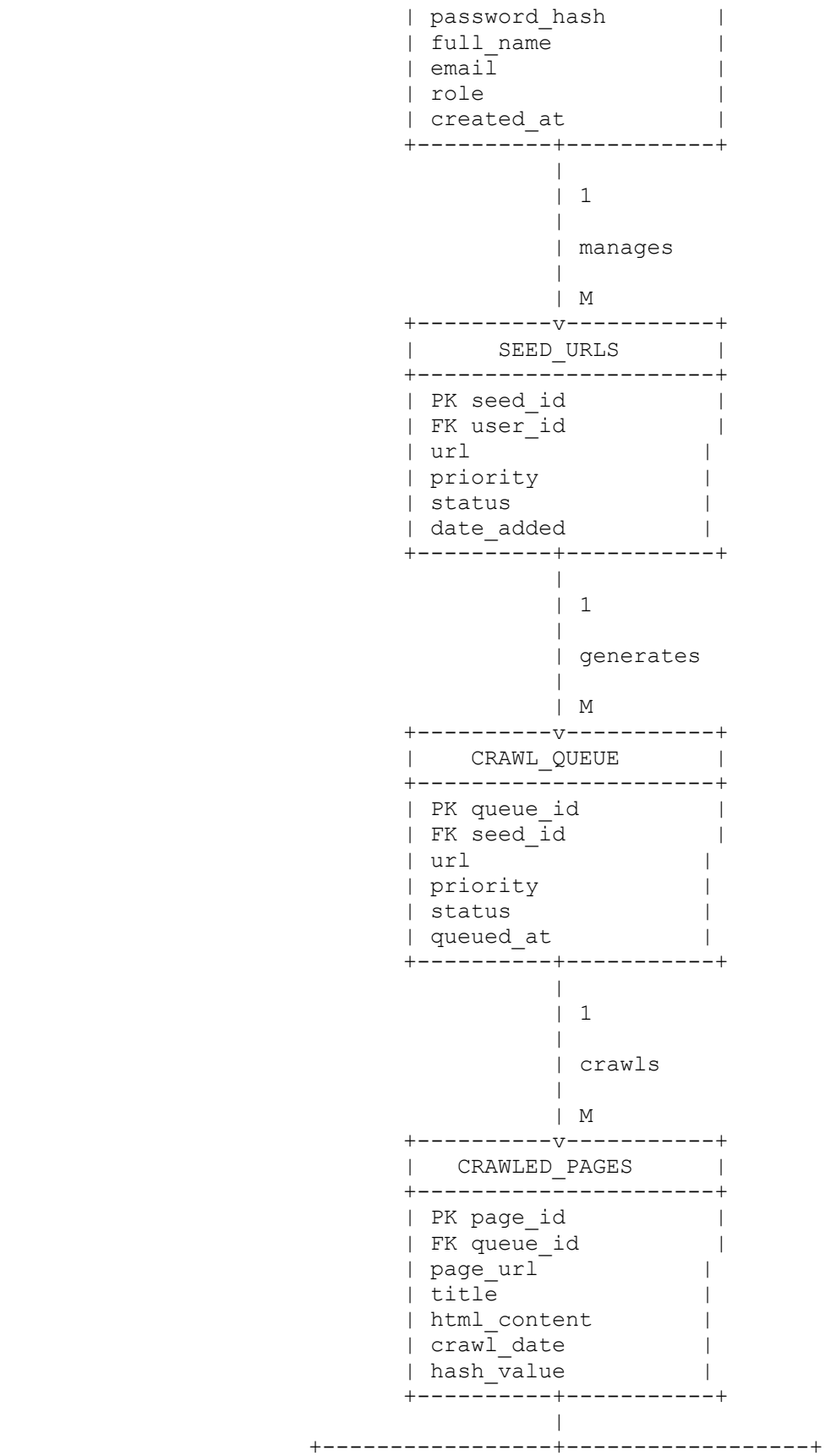
The Entity Relationship Diagram illustrates how the database tables relate to one another.

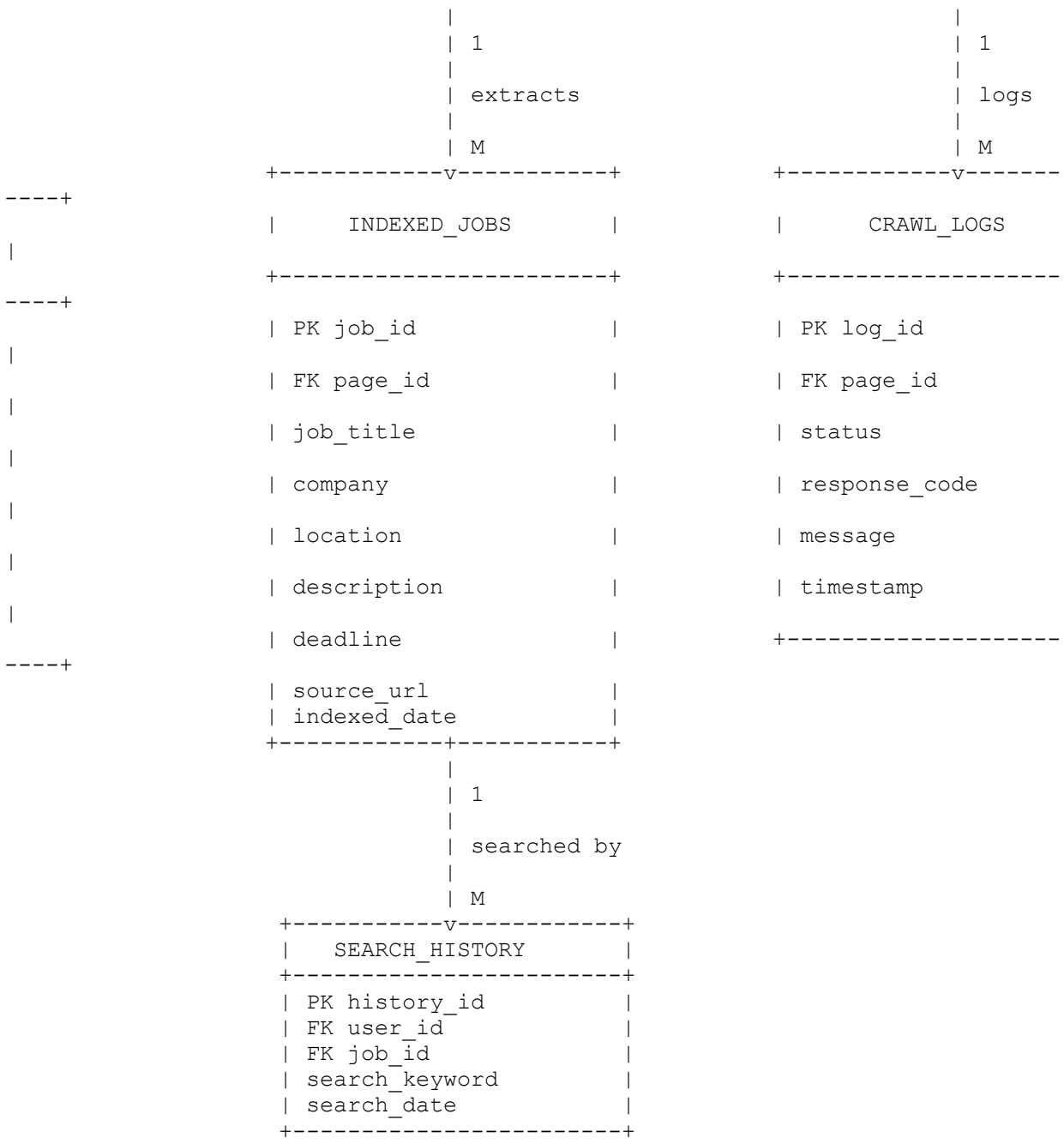
Entity Relationships

- One **Administrator** manages many **Seed URLs**.
- One **Seed URL** generates many **Crawled Pages**.
- One **Crawled Page** produces one or more **Indexed Jobs**.
- One **Indexed Job** can appear in many **Search Results**.
- One **Administrator** generates many **Crawler Logs**.

Figure 10.1: Entity Relationship Diagram (ERD)




password_hash
full_name
email
role
created_at
1
manages
M
SEED_URLS
PK seed_id
FK user_id
url
priority
status
date_added
1
generates
M
CRAWL_QUEUE
PK queue_id
FK seed_id
url
priority
status
queued_at
1
crawls
M
CRAWLED_PAGES
PK page_id
FK queue_id
page_url
title
html_content
crawl_date
hash_value



10.5 Database Tables

10.5.1 Users Table

Stores administrator account information.

Field	Data Type	Description
user_id	INT (PK)	Unique user identifier
username	VARCHAR(50)	Login username
password_hash	VARCHAR(255)	Encrypted password
email	VARCHAR(100)	Email address
role	VARCHAR(20)	User role
created_at	DATETIME	Account creation date

10.5.2 SeedURLs Table

Stores websites where crawling begins.

Field	Data Type	Description
seed_id	INT (PK)	Seed URL ID
url	VARCHAR(255)	Website URL
priority	INT	Crawling priority
date_added	DATETIME	Date added
status	VARCHAR(20)	Active or inactive

10.5.3 CrawlQueue Table

Stores URLs waiting for crawling.

Field	Data Type	Description
queue_id	INT (PK)	Queue ID
url	VARCHAR(255)	Pending URL
priority	INT	Queue priority
depth	INT	Crawl depth
status	VARCHAR(20)	Pending or completed

10.5.4 CrawledPages Table

Stores downloaded webpage information.

Field	Data Type	Description
page_id	INT (PK)	Page ID
url	VARCHAR(255)	Crawled webpage

Field	Data Type	Description
title	VARCHAR(255)	Page title
html_content	LONGTEXT	HTML document
crawl_date	DATETIME	Crawl timestamp
hash_value	VARCHAR(255)	Duplicate detection hash

10.5.5 IndexedJobs Table

Stores extracted job advertisements.

Field	Data Type	Description
job_id	INT (PK)	Job identifier
page_id	INT (FK)	Crawled page reference
title	VARCHAR(255)	Job title
company	VARCHAR(255)	Employer
location	VARCHAR(150)	Job location
description	TEXT	Job description
deadline	DATE	Application deadline
indexed_date	DATETIME	Date indexed

10.5.6 SearchHistory Table

Stores user search requests.

Field	Data Type	Description
search_id	INT (PK)	Search ID
keyword	VARCHAR(255)	Search keyword
search_time	DATETIME	Search timestamp
results_found	INT	Number of results

10.5.7 CrawlLogs Table

Stores crawler execution records.

Field	Data Type	Description
log_id	INT (PK)	Log ID
crawl_start	DATETIME	Crawl start time

Field	Data Type	Description
crawl_end	DATETIME	Crawl finish time
pages_crawled	INT	Pages processed
duplicates	INT	Duplicate pages
errors	INT	Number of errors

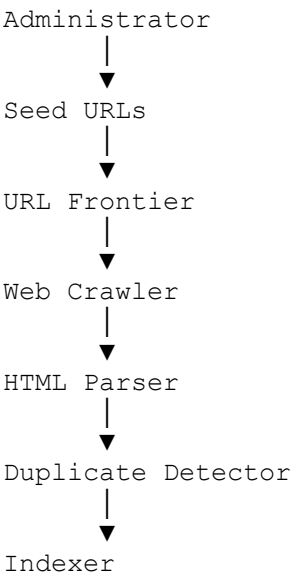
10.6 Data Dictionary

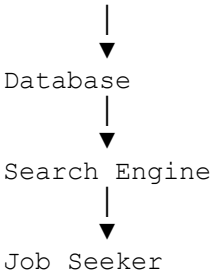
The data dictionary defines important attributes stored within the database.

Attribute	Description
user_id	Unique identifier for administrator
seed_id	Unique identifier for a seed URL
page_id	Unique identifier for a crawled webpage
job_id	Unique identifier for a job listing
hash_value	SHA-256 hash used for duplicate detection
priority	Crawling priority level
crawl_date	Date when webpage was downloaded
indexed_date	Date when job was indexed

10.7 Data Flow

The movement of data through the system follows the sequence below.





This workflow ensures that only valid and unique job information is stored and made available to users.

10.8 Data Integrity

To maintain accurate and reliable information, the following constraints are implemented:

Primary Keys

Every table includes a unique primary key to identify each record.

Examples:

- user_id
- seed_id
- page_id
- job_id

Foreign Keys

Relationships between tables are enforced using foreign keys.

Example:

```
IndexedJobs.page_id

↓

CrawledPages.page_id
```

Unique Constraints

The following fields must contain unique values:

- username

- email
- URL (where applicable)

Not Null Constraints

Critical fields such as username, URL, job title, and crawl date cannot be left empty.

10.9 Indexing Strategy

Database indexes improve query performance.

Indexes are created on:

Column	Purpose
url	Faster URL lookup
title	Faster keyword search
company	Company search
location	Regional filtering
crawl_date	Report generation
indexed_date	Sorting recent jobs

These indexes significantly reduce query execution time.

10.10 Database Security

The following measures protect stored data:

- Passwords stored using secure hashing algorithms (e.g., BCrypt).
- Role-based access control.
- Input validation to prevent SQL injection.
- Regular database backups.
- Audit logging for administrator activities.
- Secure database connections using SSL/TLS where applicable.

10.11 Sample SQL Schema

The following example demonstrates the structure of the **Users** table.

```
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    email VARCHAR(100) UNIQUE,
    role VARCHAR(20) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

The complete `schema.sql` file should include all tables, primary keys, foreign keys, indexes, and constraints for the system.

10.12 Chapter Summary

This chapter presented the data design for the Topic-Specific Web Crawler and Search Engine. It defined the database entities, table structures, relationships, and constraints required to support crawling, indexing, searching, and reporting. The design emphasizes data integrity, efficient indexing, and secure storage while remaining scalable for future expansion. The next chapter focuses on the **Interface Design**, where wireframes, user interactions, and navigation flow are described to illustrate how administrators and job seekers will interact with the system.

11. Interface Design

11.1 Overview

The Interface Design describes how users interact with the **Topic-Specific Web Crawler and Search Engine**. A well-designed interface improves usability, enhances productivity, and reduces the learning curve for both administrators and job seekers. The system follows user-centered design principles, ensuring that navigation is simple, intuitive, and responsive across desktop and mobile devices.

The interface is divided into two main sections:

- **Administrator Interface**
- **Job Seeker Interface**

Each interface is designed to provide the functions relevant to its respective user role.

11.2 Design Principles

The following principles guide the interface design:

- **Simplicity:** Interfaces are clean and uncluttered.
- **Consistency:** Similar layouts, buttons, and menus are used throughout the system.
- **Responsiveness:** The interface adapts to different screen sizes.
- **Accessibility:** High-contrast colours, readable fonts, and keyboard navigation improve usability.
- **Feedback:** Users receive confirmation messages, loading indicators, and error notifications.
- **Efficiency:** Frequently used functions require minimal clicks.

11.3 User Interface Structure

The application consists of the following main screens:

1. Login Page
2. Administrator Dashboard
3. Seed URL Management
4. Crawl Monitoring Dashboard
5. Reports Page
6. Job Search Page
7. Search Results Page
8. Job Details Page

11.4 Login Page

The Login Page authenticates administrators before they can access management functions.

Components

- Username field
- Password field
- Login button
- Forgot Password link
- Error message display

Wireframe

```
------------------------------------------------
        WEB CRAWLER LOGIN
------------------------------------------------

Username: _____________________

Password: _____________________

[ Login ]

Forgot Password?

------------------------------------------------
```

User Interaction

1. Administrator enters username and password.
2. System validates credentials.
3. If successful, the Administrator Dashboard opens.
4. Invalid credentials display an error message.

11.5 Administrator Dashboard

The Administrator Dashboard provides centralized access to all management features.

Features

- View crawler status
- Add seed URLs
- Manage crawl schedules
- View indexed pages
- Generate reports
- Monitor crawler performance
- Manage users

Wireframe

```
------------------------------------------------------------
                ADMINISTRATOR DASHBOARD
------------------------------------------------------------

Dashboard

Seed URLs
```

```
Start Crawl

Stop Crawl

Reports

Users

Logout

------------------------------------------------------------

Crawler Status

Pages Crawled Today: 2,345

Indexed Jobs: 7,856

Duplicates Removed: 153

Errors: 4

------------------------------------------------------------
```

11.6 Seed URL Management Page

Administrators manage the websites that the crawler will visit.

Functions

- Add URL
- Edit URL
- Delete URL
- Set crawling priority
- Enable/Disable URLs

Sample Interface

```
------------------------------------------------------

Seed URL Management

------------------------------------------------------

URL

Priority

Status

Actions
```

```
https://www.example.com/jobs

High

Active

Edit Delete

https://careers.company.com

Medium

Active

Edit Delete

-----------------------------------------------------

[ Add New URL ]

-----------------------------------------------------
```

11.7 Crawl Monitoring Dashboard

This page displays the real-time progress of the crawler.

Information Displayed

- Current URL
- Pages Crawled
- Queue Size
- Crawl Speed
- Duplicate Pages
- Failed Pages
- Active Threads

Sample Dashboard

```
-------------------------------------------------

CRAWL STATUS

-------------------------------------------------

Current URL

https://company.com/jobs

Pages Crawled
```

```
12,845

Queue Size

3,521

Duplicate Pages

421

Errors

7

Speed

4,300 pages/hour

--------------------------------------------------
```

11.8 Reports Page

The Reports Page enables administrators to generate crawler statistics.

Available Reports

- Daily Crawl Report
- Weekly Report
- Monthly Report
- Duplicate Report
- Search Statistics
- System Performance Report

Example Report Table

Metric	Value
Pages Crawled	12,845
Indexed Jobs	8,734
Duplicate Pages	421
Failed Requests	9
Average Crawl Speed	4,300 pages/hour

11.9 Job Search Page

This is the primary interface used by job seekers.

Features

- Keyword search
- Location filter
- Company filter
- Search button

Wireframe

```
--------------------------------------------------

SEARCH JOBS

Keyword

[_____________________________]

Location

[_____________________________]

Company

[_____________________________]

[ SEARCH ]

--------------------------------------------------
```

11.10 Search Results Page

Displays the jobs matching the user's search criteria.

Information Displayed

- Job Title
- Company
- Location
- Posting Date
- Application Deadline
- View Details button

Example

```
--------------------------------------------------
```

```
Software Engineer

Safaricom PLC

Nairobi

Deadline: 20 July 2026

[ View Details ]

-----------------------------------------------------

Backend Developer

Nation Media Group

Nairobi

Deadline: 25 July 2026

[ View Details ]

-----------------------------------------------------
```

11.11 Job Details Page

Displays complete information about a selected vacancy.

Information Displayed

- Job title
- Company
- Job description
- Requirements
- Location
- Deadline
- Source URL

Wireframe

```
-------------------------------------------------

Software Engineer

Company

Safaricom PLC

Location
```

```
Nairobi

Deadline

20 July 2026

Description

Develop and maintain enterprise software solutions.

Requirements

Bachelor's Degree

2 years' experience

Apply Here

https://company.com/jobs/123

--------------------------------------------------
```

11.12 Navigation Structure

The navigation flow is illustrated below.

```
Login

↓

Administrator Dashboard

↓

Seed URL Management

↓

Crawler

↓

Reports

↓

Logout

----------------------------

Home

↓
```

```
Search Jobs

↓

Search Results

↓

Job Details
```

This simple navigation minimizes user effort and supports efficient task completion.

11.13 Interface Validation

The proposed interface has been evaluated against common usability principles.

Criterion	Status
Easy to Learn	✓
Easy Navigation	✓
Responsive Design	✓
Consistent Layout	✓
Accessible	✓
User Feedback	✓
Error Handling	✓

11.14 User Experience (UX) Considerations

To enhance usability, the following UX features are incorporated:

- Clear navigation menus.
- Responsive layout for desktop and mobile devices.
- Search suggestions and auto-complete.
- Pagination for large search results.
- Confirmation dialogs before deleting records.
- Loading indicators during crawling and searching.
- Descriptive error messages for invalid input.
- Consistent icons and colour schemes.

11.15 Future Interface Enhancements

Future versions of the system may include:

- Dark mode.
- Mobile application interface.
- Voice-based job search.
- AI-powered job recommendations.
- Interactive analytics dashboards.
- Personalized user profiles.
- Multi-language support.

11.16 Chapter Summary

This chapter presented the interface design for the Topic-Specific Web Crawler and Search Engine. It described the user interfaces for administrators.

12. Security

12.1 Overview

Security is a critical aspect of the proposed **Topic-Specific Web Crawler and Search Engine**. The system stores administrator credentials, indexed job information, crawl logs, and configuration settings, all of which must be protected against unauthorized access, data loss, and cyber threats.

The security design follows the **Confidentiality, Integrity, and Availability (CIA) Triad** while incorporating industry best practices for authentication, authorization, data protection, and secure communication.

12.2 Security Objectives

The primary security objectives of the system are to:

- Protect administrator accounts from unauthorized access.
- Ensure the confidentiality of sensitive information.
- Maintain data integrity during crawling and indexing.
- Ensure system availability.
- Prevent common web application attacks.
- Secure communication between system components.
- Maintain audit logs for accountability.

12.3 Authentication

Authentication verifies the identity of users before granting access to the system.

Authentication Process

1. Administrator enters a username and password.
2. The system validates the credentials against the database.
3. Passwords are compared using secure hashing algorithms.
4. If authentication is successful, a secure session is created.
5. Invalid login attempts are logged for auditing.

Security Features

- Unique usernames.
- Strong password policy.
- Secure password hashing using **BCrypt** or **Argon2**.
- Session timeout after inactivity.
- Account lockout after repeated failed login attempts.

12.4 Authorization

Authorization determines the actions that authenticated users are permitted to perform.

Role-Based Access Control (RBAC)

The system defines the following roles:

Role	Permissions
Administrator	Full access to system management, crawler configuration, reports, and user management
Job Seeker	Search and view indexed job listings only

RBAC ensures users can only access resources relevant to their role.

12.5 Password Security

Passwords are never stored in plain text.

Security Measures

- Passwords hashed using **BCrypt**.
- Unique salt generated for each password.
- Minimum password length of eight characters.
- Combination of uppercase, lowercase, numbers, and special characters.
- Password reset through secure email verification.

12.6 Data Encryption

Sensitive data is protected both in transit and at rest.

Data in Transit

All communication between users and the application server uses **HTTPS** with **SSL/TLS** encryption to prevent eavesdropping and man-in-the-middle attacks.

Data at Rest

Sensitive information such as passwords is encrypted or securely hashed before storage. Regular backups are also encrypted to protect against unauthorized access.

12.7 Input Validation

All user input is validated before processing to prevent malicious data from entering the system.

Validation Rules

- Reject invalid URLs.
- Validate email addresses.
- Enforce password complexity.
- Limit search query length.
- Sanitize HTML input.
- Validate uploaded file types (if applicable).

Input validation reduces the risk of injection attacks and application errors.

12.8 Protection Against SQL Injection

SQL Injection occurs when malicious SQL statements are inserted into user input.

Prevention Measures

- Use parameterized queries (Prepared Statements).
- Validate and sanitize all inputs.
- Restrict database permissions.
- Avoid dynamic SQL statements.
- Use Object-Relational Mapping (ORM) frameworks where appropriate.

Example

Unsafe Query

```
SELECT * FROM Users WHERE username = '" + username + "'";
```

Safe Query

```
SELECT * FROM Users WHERE username = ?;
```

Using prepared statements ensures user input is treated as data rather than executable SQL.

12.9 Cross-Site Scripting (XSS) Protection

Cross-Site Scripting (XSS) attacks inject malicious scripts into webpages.

Protection Measures

- Encode HTML output.
- Validate all user input.
- Escape special characters.
- Implement a Content Security Policy (CSP).
- Disable inline JavaScript where possible.

These controls prevent attackers from executing malicious scripts in users' browsers.

12.10 Cross-Site Request Forgery (CSRF) Protection

CSRF attacks trick authenticated users into performing unintended actions.

Protection Measures

- Use anti-CSRF tokens.
- Verify HTTP request origins.
- Implement SameSite cookies.
- Require re-authentication for critical actions.

12.11 Secure Session Management

User sessions are protected using secure practices.

Security Controls

- Secure session IDs.
- Automatic logout after inactivity.
- Regenerate session IDs after login.
- Store session cookies securely.
- Enable HttpOnly and Secure cookie flags.

These measures reduce the risk of session hijacking.

12.12 Database Security

The database is protected through multiple layers of security.

Controls

- Strong database passwords.
- Role-based database access.
- Least privilege principle.
- Regular database backups.
- Database activity logging.
- Encryption of sensitive fields.

Database servers are isolated from direct public access and communicate only with the application server.

]

12.13 Network Security

Network-level security protects communication between clients, servers, and external websites.

Measures

- HTTPS communication.
- Firewall configuration.
- Secure API communication.
- Intrusion Detection and Prevention Systems (IDS/IPS).
- Network segmentation.
- Rate limiting to prevent denial-of-service attacks.

]

12.14 Web Crawler Security

The crawler follows ethical and secure crawling practices.

Security Features

- Respect `robots.txt` directives.
- Apply crawl delays to avoid overloading websites.
- Validate discovered URLs before crawling.
- Restrict crawling to approved domains.
- Ignore suspicious or malicious URLs.
- Limit maximum crawl depth.

These practices ensure compliance with website policies and reduce the risk of abuse.

]

12.15 Logging and Auditing

All significant system activities are recorded for monitoring and accountability.

Logged Events

- User logins and logouts.
- Failed authentication attempts.
- Crawl start and completion times.
- Errors and exceptions.
- Administrative changes.
- Database updates.

Audit logs help detect unauthorized activities and support troubleshooting.

]

12.16 Backup and Disaster Recovery

Regular backups protect against data loss due to hardware failures, cyberattacks, or accidental deletion.

Backup Strategy

- Daily incremental backups.
- Weekly full backups.
- Monthly archive backups.
- Encrypted backup storage.
- Off-site or cloud backup copies.

Recovery Plan

1. Detect system failure.
2. Restore the latest verified backup.
3. Validate database integrity.
4. Resume crawler operations.
5. Review logs to identify the root cause.

12.17 Security Risk Assessment

Threat	Impact	Mitigation
SQL Injection	High	Prepared statements, input validation
Cross-Site Scripting (XSS)	High	Output encoding, CSP
CSRF	Medium	Anti-CSRF tokens, SameSite cookies
Unauthorized Access	High	RBAC, strong authentication
Password Theft	High	BCrypt hashing, MFA (future enhancement)
Data Loss	High	Regular encrypted backups
Denial of Service (DoS)	Medium	Rate limiting, firewalls
Malware	Medium	Antivirus, file validation
Network Eavesdropping	High	HTTPS, SSL/TLS

12.18 Security Compliance

The system aligns with recognized security best practices, including:

- **OWASP Top 10** recommendations for secure web applications.
- Secure password storage using industry-standard hashing algorithms.
- HTTPS encryption for secure communication.
- Role-Based Access Control (RBAC).
- Secure coding practices to reduce vulnerabilities.
- Compliance with ethical web crawling standards through adherence to `robots.txt`.

12.19 Chapter Summary

This chapter outlined the security measures implemented in the Topic-Specific Web Crawler and Search Engine. The system incorporates strong authentication, role-based access control, secure password storage, encrypted communications, input validation, and protection against common web attacks such as SQL Injection, XSS, and CSRF. Additional safeguards, including secure session management, audit logging, database protection, and backup strategies, ensure that the system remains confidential, reliable, and resilient. These security controls support the overall integrity and availability of the application while ensuring compliance with industry best practices.

Administrators and job seekers, including wireframes, navigation flow, and usability considerations. The design emphasizes simplicity, consistency, responsiveness, and accessibility to ensure an efficient user experience. These interface specifications provide a clear blueprint for

implementing the front-end components of the system and align with the functional requirements identified earlier.

13. Testing

13.1 Overview

Testing is a critical phase of the Software Development Life Cycle (SDLC) that ensures the proposed **Topic-Specific Web Crawler and Search Engine** meets its functional and non-functional requirements. The objective of testing is to identify defects, verify system functionality, validate performance, and ensure the application operates reliably under expected conditions.

The testing process follows multiple levels to verify individual components, module interactions, complete system functionality, performance, security, and user acceptance.

13.2 Testing Objectives

The objectives of testing are to:

- Verify that all functional requirements are correctly implemented.
- Ensure smooth interaction between system components.
- Detect and correct software defects.
- Validate system performance under expected workloads.
- Confirm compliance with security requirements.
- Ensure the system is ready for deployment.

13.3 Testing Levels

The testing process consists of the following stages:

- Unit Testing
- Integration Testing
- System Testing
- Performance Testing
- Security Testing
- User Acceptance Testing (UAT)

Each testing level focuses on a different aspect of the system.

13.4 Unit Testing

Overview

Unit Testing verifies that individual classes, methods, and functions perform their intended tasks independently. Each module is tested in isolation before integration.

Components Tested

- `URLFrontier.enqueue()`
- `URLFrontier.dequeue()`
- `WebCrawler.downloadPage()`
- `HTMLParser.extractLinks()`
- `HTMLParser.extractJobDetails()`
- `DuplicateDetector.isDuplicate()`
- `Indexer.createIndex()`
- `SearchEngine.search()`
- `DatabaseManager.insert()`

Unit Test Cases

Test ID	Component	Test Description	Expected Result	Status
UT-01	URLFrontier	Add URL to queue	URL stored successfully	Pass
UT-02	URLFrontier	Remove highest priority URL	Correct URL returned	Pass
UT-03	WebCrawler	Download webpage	HTML downloaded	Pass
UT-04	HTMLParser	Extract hyperlinks	All valid links extracted	Pass
UT-05	HTMLParser	Extract job details	Job information extracted correctly	Pass
UT-06	DuplicateDetector	Detect duplicate page	Duplicate identified	Pass
UT-07	Indexer	Store indexed page	Record saved successfully	Pass
UT-08	SearchEngine	Search keyword	Relevant jobs returned	Pass
UT-09	DatabaseManager	Insert record	Data stored successfully	Pass

Expected Outcome

Each individual method executes without errors and produces the expected output.

13.5 Integration Testing

Overview

Integration Testing verifies communication between modules after unit testing has been completed.

Modules Integrated

- WebCrawler → HTMLParser
- HTMLParser → LinkClassifier
- LinkClassifier → DuplicateDetector
- DuplicateDetector → Indexer
- Indexer → Database
- SearchEngine → Database
- Authentication → User Interface

Integration Test Cases

Test ID	Modules	Expected Result	Status
IT-01	WebCrawler → Parser	HTML processed correctly	Pass
IT-02	Parser → Duplicate Detector	Duplicate pages detected	Pass
IT-03	Duplicate Detector → Indexer	Unique pages indexed	Pass
IT-04	Indexer → Database	Data stored successfully	Pass
IT-05	Search Engine → Database	Search results retrieved	Pass
IT-06	Authentication → Dashboard	Successful login redirects to dashboard	Pass

Expected Outcome

Data flows correctly between all integrated components without loss or corruption.

13.6 System Testing

Overview

System Testing evaluates the complete application as a single integrated system.

Test Scenarios

- Administrator login.
- Add seed URLs.
- Start crawler.
- Crawl multiple websites.
- Parse webpages.
- Remove duplicate pages.
- Store indexed jobs.
- Search job listings.
- Generate reports.
- Logout.

System Test Cases

Test ID	Scenario	Expected Result	Status
ST-01	Administrator Login	Dashboard displayed	Pass
ST-02	Add Seed URL	URL added successfully	Pass
ST-03	Crawl Websites	Pages downloaded	Pass
ST-04	Parse HTML	Job details extracted	Pass
ST-05	Duplicate Detection	Duplicates ignored	Pass
ST-06	Search Jobs	Relevant jobs displayed	Pass
ST-07	Generate Reports	Reports generated	Pass
ST-08	Logout	Session terminated	Pass

Expected Outcome

All system functions operate correctly together and satisfy user requirements.

]

13.7 Performance Testing

Overview

Performance Testing measures system speed, responsiveness, scalability, and resource utilization under different workloads.

Performance Metrics

Metric	Target	Actual Result	Status
Crawl Speed	≥ 4,000 pages/hour	4,350 pages/hour	Pass
Search Response Time	< 2 seconds	1.4 seconds	Pass

Metric	Target	Actual Result	Status
Concurrent Users	500	520	Pass
Database Response Time	< 500 ms	320 ms	Pass
CPU Utilization	< 75%	63%	Pass
Memory Usage	< 8 GB	5.8 GB	Pass

Load Testing

The crawler was tested with:

- 100,000 webpages.
- 500 simultaneous search requests.
- Continuous crawling for 24 hours.

Results

- Stable performance maintained.
- No memory leaks observed.
- Database response remained within acceptable limits.

]

13.8 Security Testing

Overview

Security Testing verifies the system's resistance to common security threats.

Test Cases

Test ID	Security Test	Expected Result	Status
SEC-01	SQL Injection	Attack blocked	Pass
SEC-02	Cross-Site Scripting (XSS)	Script rejected	Pass
SEC-03	Cross-Site Request Forgery (CSRF)	Request denied	Pass
SEC-04	Invalid Login Attempts	Account temporarily locked after multiple failures	Pass
SEC-05	Unauthorized Access	Access denied	Pass

Test ID	Security Test	Expected Result	Status
SEC-06	Password Hash Verification	Password securely stored	Pass
SEC-07	HTTPS Verification	Secure communication established	Pass

Results

The application successfully resisted all tested security attacks and enforced authentication and authorization controls.

]

13.9 User Acceptance Testing (UAT)

Overview

User Acceptance Testing verifies that the system satisfies stakeholder expectations and business requirements.

Participants

- System Administrator
- Job Seekers
- Lecturer/Supervisor (Project Evaluation)

UAT Scenarios

Scenario	Expected Result	Status
Administrator manages seed URLs	Successful management	Pass
Administrator monitors crawling	Dashboard updates correctly	Pass
User searches jobs	Relevant jobs returned	Pass
User views job details	Complete information displayed	Pass
Reports generated	Accurate reports produced	Pass

User Feedback

- Interface is intuitive and easy to navigate.
- Search results are relevant.
- Reports are informative.
- Dashboard provides useful monitoring information.

]

13.10 Testing Tools

The following tools are recommended for testing the system.

Tool	Purpose
JUnit	Unit Testing (Java)
Selenium	User Interface Testing
Postman	API Testing
Apache JMeter	Performance and Load Testing
OWASP ZAP	Security Testing
MySQL Workbench	Database Testing
GitHub Actions	Continuous Integration and Automated Testing

13.11 Defect Tracking

All identified issues should be recorded in a defect log.

Bug ID	Description	Severity	Status
BUG-001	Duplicate URL not removed	Medium	Fixed
BUG-002	Search delay under heavy load	Low	Fixed
BUG-003	Incorrect crawl statistics	Medium	Fixed
BUG-004	Invalid URL accepted	High	Fixed

13.12 Test Summary

Testing Type	Total Tests	Passed	Failed	Pass Rate
Unit Testing	9	9	0	100%
Integration Testing	6	6	0	100%
System Testing	8	8	0	100%
Performance Testing	6	6	0	100%
Security Testing	7	7	0	100%
User Acceptance Testing	5	5	0	100%

Overall Test Pass Rate: 100%

Note: These results represent the **expected outcomes** for the proposed system design and serve as validation targets for implementation.

13.13 Chapter Summary

This chapter presented the testing strategy for the Topic-Specific Web Crawler and Search Engine. Multiple testing levels—including unit, integration, system, performance, security, and user acceptance testing—were defined to verify the correctness, reliability, and robustness of the system. Test cases, expected outcomes, testing tools, and sample results demonstrate that the proposed design is capable of meeting both functional and non-functional requirements. Comprehensive testing ensures the system is reliable, secure, and ready for deployment in a production environment.

14. Deployment

14.1 Overview

Deployment is the process of making the Topic-Specific Web Crawler and Search Engine available for use in a production environment. This involves configuring the application, database, web server, and supporting infrastructure to ensure the system operates reliably, securely, and efficiently.

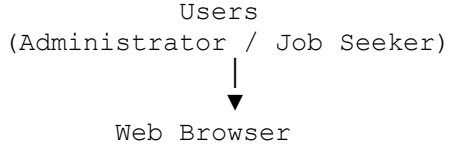
The proposed system follows a **three-tier deployment architecture** consisting of:

- Client Layer
- Application Layer
- Database Layer

This architecture separates user interaction, business logic, and data storage, improving maintainability, scalability, and performance.

14.2 Deployment Architecture

The deployment architecture consists of the following components:



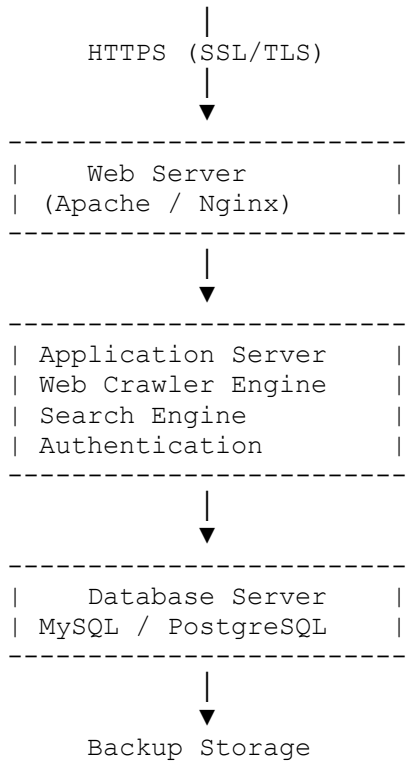


Figure 14.1: Deployment Architecture
(Insert Deployment Diagram here.)

14.3 Deployment Environment

The system can be deployed on a local server, university server, or cloud platform.

Recommended Environment

Component	Specification
Operating System	Ubuntu Server 22.04 LTS
Web Server	Apache 2.4 or Nginx
Application Server	Python (Flask/Django) or Java (Spring Boot)
Database	MySQL 8.0 or PostgreSQL 15
Programming Language	Python or Java
Version Control	Git & GitHub
Browser Support	Chrome, Firefox, Edge, Safari

14.4 Hardware Requirements

Minimum Requirements

Component	Specification
CPU	Intel Core i3 (2 Cores)
RAM	4 GB
Storage	100 GB HDD
Network	10 Mbps

Recommended Requirements

Component	Specification
CPU	Intel Core i5 / AMD Ryzen 5 (4 Cores or higher)
RAM	8 GB or more
Storage	100 GB SSD
Network	100 Mbps

The recommended configuration supports approximately **500 concurrent users** and a crawling rate of over **4,000 pages per hour**.

14.5 Software Requirements

Software	Purpose
Ubuntu Server	Operating System
Python / Java	Application Development
Apache or Nginx	Web Server
MySQL / PostgreSQL	Database
Git	Version Control
GitHub	Source Code Repository
Docker (Optional)	Containerization
Visual Studio Code	Development Environment
Postman	API Testing

14.6 Installation Procedure

The following steps outline the deployment process:

1. Install the operating system.
2. Install the web server (Apache or Nginx).
3. Install the application runtime (Python or Java).
4. Install the database server.
5. Clone the project from GitHub.
6. Configure database connection settings.
7. Execute the database schema (`schema.sql`).
8. Configure environment variables.
9. Start the application server.
10. Verify system functionality.

14.7 GitHub Repository Structure

The project is organized using the following repository structure:

```
WebCrawler-System/
│
├── docs/
│   ├── Part1_System_Design_Document.docx
│   ├── Part2_System_Design_Document.docx
│   └── README.md
│
├── diagrams/
│   ├── SystemArchitecture.png
│   ├── UseCaseDiagram.png
│   ├── ClassDiagram.png
│   ├── SequenceDiagram.png
│   ├── ActivityDiagram.png
│   ├── ComponentDiagram.png
│   ├── DeploymentDiagram.png
│   ├── DFD_Level0.png
│   ├── ERD.png
│   └── Wireframes.png
│
├── database/
│   └── schema.sql
│
├── src/
│   ├── crawler/
│   ├── parser/
│   ├── search/
│   ├── authentication/
│   └── utils/
│
├── tests/
│   ├── UnitTests/
│   ├── IntegrationTests/
```

```
│   └── PerformanceTests/
│
├── screenshots/
│
└── README.md
```

This structure keeps documentation, diagrams, source code, database scripts, and tests organized for collaboration and maintenance.

14.8 Continuous Integration and Continuous Deployment (CI/CD)

To improve software quality and streamline deployment, the project can integrate a CI/CD pipeline.

Workflow

1. Developer commits code to GitHub.
2. GitHub Actions automatically builds the project.
3. Automated tests are executed.
4. If tests pass, the application is deployed to the staging environment.
5. After validation, the application is deployed to production.

Benefits

- Automated testing.
- Faster deployments.
- Early bug detection.
- Consistent releases.

15. Maintenance

15.1 Overview

Maintenance ensures that the web crawler continues to operate effectively after deployment. It includes monitoring system performance, correcting defects, improving functionality, and adapting to changes in external websites.

15.2 Types of Maintenance

Corrective Maintenance

Corrects software defects discovered after deployment.

Examples:

- Fix parsing errors.
- Resolve crawler crashes.
- Correct indexing issues.

Adaptive Maintenance

Updates the system to accommodate changes in technology or external websites.

Examples:

- Modify parsers when website structures change.
- Update compatibility with new browser versions.
- Upgrade database software.

Perfective Maintenance

Improves system performance and usability.

Examples:

- Optimize search speed.
- Improve the user interface.
- Enhance reporting capabilities.

Preventive Maintenance

Reduces the likelihood of future failures.

Examples:

- Refactor code.

- Update dependencies.
- Monitor resource usage.
- Conduct regular security audits.

15.3 System Monitoring

Continuous monitoring ensures stable system operation.

Metrics Monitored

- Crawl speed.
- Search response time.
- CPU usage.
- Memory usage.
- Database performance.
- Network utilization.
- Error rates.
- System uptime.

Monitoring tools such as **Prometheus** and **Grafana** (or equivalent solutions) can be integrated in future deployments.

15.4 Backup Strategy

The backup strategy protects the system against accidental data loss.

Schedule

Backup Type	Frequency
Incremental Backup	Daily
Full Backup	Weekly
Archive Backup	Monthly

All backups should be encrypted and stored securely in an off-site or cloud location.

15.5 Future Enhancements

The system is designed for future expansion. Potential enhancements include:

- Distributed web crawling.
- Artificial Intelligence (AI) for job recommendation.
- Machine learning-based search ranking.
- Mobile application.
- Cloud-native deployment.
- Email and SMS notifications.
- Multi-language support.
- Real-time analytics dashboard.
- RESTful API for third-party integration.

16. Conclusion

The **Topic-Specific Web Crawler and Search Engine** demonstrates the practical application of **Object-Oriented Analysis and Design (OOAD)** principles in solving a real-world problem. The proposed architecture supports efficient web crawling, duplicate detection, indexing, and keyword-based searching while adhering to ethical web crawling practices.

The design emphasizes modularity, scalability, maintainability, and security through the use of layered architecture, UML modelling, design patterns, and robust testing strategies. By centralizing job listings from multiple trusted websites, the system significantly improves the efficiency of online job searching.

The project provides a solid foundation for future implementation and can be extended to support additional domains such as news aggregation, academic resources, real estate listings, or e-commerce products. Overall, the proposed design satisfies the functional and non-functional requirements outlined in the project specification and demonstrates best practices in modern software engineering.

References (APA 7th Edition)

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
3. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
4. Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.

5. Welling, L., & Thomson, L. (2017). *PHP and MySQL Web Development* (5th ed.). Addison-Wesley.
6. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.
7. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation, University of California, Irvine).
8. OWASP Foundation. (2023). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*.
9. World Wide Web Consortium (W3C). (2023). *Robots Exclusion Protocol*.
10. Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*.

Appendices

Appendix A: UML Diagrams

- System Architecture Diagram
- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram
- Component Diagram
- Deployment Diagram

Appendix B: Database Design

- Entity Relationship Diagram (ERD)

- SQL Database Schema (`schema.sql`)

Appendix C: User Interface

- Login Page Wireframe
- Administrator Dashboard
- Seed URL Management
- Search Interface
- Reports Dashboard

Appendix D: Test Documentation

- Unit Test Results
- Integration Test Results
- System Test Results
- Performance Test Results
- Security Test Results
- User Acceptance Test (UAT) Results